

Beyond Proof of Possession: A Relational Theory of Authority

Nicola Gallo

25 September 2026

Abstract

Proof-of-Possession (PoP) establishes evidence that an executor controls an artifact such as a key, token, credential, or capability. It may support holder binding, authentication, profile evidence, or a concrete continuation protocol, but control of an artifact alone determines neither the executor’s relational profile nor the causal history of an execution occurrence. We separate PoP from Proof of Relationship (PoR), the semantic property that one execution occurrence validly continues another, and from authorization. Authorization is occurrence-sensitive and indexed by the occurrence’s causal history: a transition is evaluated from PoR, non-expansion of its authority context, satisfaction and refinement of its relational envelope, and soundly established executor profiles. The foundational change is a change in the semantic domain of authorization: from holders and authority-bearing artifacts alone to execution occurrences situated in causal histories.

Building on Provenance Identity Continuity (PIC) and Proof-of-Continuity (PoC), we distinguish execution steps from the executors that realize them and classify executors by authorization-relevant equivalence relations. Their quotients form relational profile spaces, while positive predicates over profile transitions form a distributive lattice ordered by implication. This adds relational attenuation to authority attenuation: along one valid causal history, $C_{i+1} \subseteq C_i$ and $\rho_{i+1} \Rightarrow \rho_i$. Causal history grows by prefix extension while authority and relational freedom shrink. We prove dual monotonicity, combined closure, and origin relational closure.

Because authorization is defined over occurrences situated in causal histories, two occurrences of the same authenticated executor, with the same profile and possession evidence, need not be authorization-equivalent. Within a valid PIC lineage, independent authority held by a deputy cannot be imported as a valid exercise outside the origin context. Excluding request-caused effects presented outside that lineage additionally requires complete mediation and causal attribution.

Finally, we prove that the executor-dependent decision factors through the relational profile and is invariant under sound mechanisms that establish the same profile. Successor-side PoP is therefore a possible evidence

mechanism, not a necessary semantic premise of authorization. Conditional on explicit Witness Assumptions, a boundary-witnessed checkpoint construction realizes the premises without successor-side PoP. Boundary signatures provide checkpoint evidence, profile evidence may rely on additional trust roots, and the construction does not independently establish its environmental assumptions.

1 Introduction

Most deployed authorization mechanisms attach authority to something that an actor possesses or controls: a bearer token, a sender-constrained token, a key, a credential, a capability, or an ambient privilege. Such mechanisms are often effective local authorization tools. Their common abstraction, however, is spatial: the decision is made from authority that is available to the actor at the point of use.

Proof-of-Continuity [1] introduced a different axis. A later execution step is valid only if it is causally related to its predecessor and carries no more authority than the predecessor received. Formally, for a causal chain

$$s_0 \rightarrow s_1 \rightarrow \cdots \rightarrow s_n,$$

each valid hop satisfies

$$\text{PoR}(s_i, s_{i+1}) \quad \text{and} \quad C_{i+1} \subseteq C_i,$$

where $C_i \subseteq O \times R$ is the authority context carried at step s_i [1]. This gives

$$C_n \subseteq C_{n-1} \subseteq \cdots \subseteq C_0$$

and therefore origin-bounded authority: a privilege absent from C_0 cannot appear later in the same valid lineage.

That result leaves a further question open. If PoR is explicitly a *relationship* between execution occurrences, what mathematical structure does that relationship carry? More specifically, can authorization be expressed over relational characteristics of the execution rather than over possession by a particular holder?

The foundational change is not a new credential format but a change in the domain over which authorization is defined: from a decision described only by holders and authority-bearing artifacts to one evaluated over execution occurrences situated in causal histories. Two occurrences

of the same authenticated executor, presenting the same possession evidence, need not be authorization-equivalent when they occur in different histories. The same agent may present the same key, token and profile in two requests; a write may be admissible in the history whose origin context contains it and inadmissible in the history whose origin context does not. The difference is not the holder or artifact but the causal history of the occurrence.

Thus possessing authority is not the same as being the execution occurrence permitted to exercise it in the current causal history. Bare possession does not determine $\text{Lineage}(e)$ or $\text{PoR}(s_i, s_{i+1})$. Authority is exercisable not merely because an executor possesses it, but because the current occurrence is an admitted continuation of a history in which that authority remains valid.

To answer the question above, this paper introduces a relational space for executors. Let X be the set of executor instances. An authorization-relevant equivalence relation $\sim_j$ partitions X into classes of executors that are indistinguishable with respect to one relational characteristic. The quotient

$$\mathcal{R}_j := X / \sim_j$$

therefore forgets all executor differences irrelevant to that characteristic. Different relations induce different quotient views, so the same executor may belong to different equivalence classes under different relations.

We then define relational predicates over combinations of these quotient views. Predicates compose with logical operators such as $\wedge$ and $\vee$, and logical implication induces an attenuation order:

$$\rho' \preceq_R \rho \iff \rho' \Rightarrow \rho.$$

Thus a successor may preserve or restrict a relational authorization envelope, but may not expand it.

The result is a dual monotonicity. Along one causal chain,

$$C_{i+1} \subseteq C_i$$

restricts authority, while

$$\rho_{i+1} \Rightarrow \rho_i$$

restricts relational admissibility. The lineage prefix itself moves in the opposite direction: causal history accumulates. Hence continuity has a simple structural form:

history grows, while authority
and relational freedom shrink.

The paper makes six contributions:

1. it separates execution steps from executors and gives executor relations a quotient-space semantics;
2. it defines multiple relational views of the same executor and a combined relational profile space;
3. it defines a positive relational predicate lattice with refinement ordered by implication;

4. it proves dual monotonicity and relational closure along a PoR-linked causal chain;
5. it shows that the lineage-forgetting projection of the previous paper is itself a relational quotient, so that its lineage-invariance result for possession-based policies is an instance of the factorization property developed here; and
6. it proves that the executor-dependent component of authorization factors through the relational profile, formalizes invariance under sound profile-evidence mechanisms that produce the same profile, and gives, relative to explicit Witness Assumptions, a construction in which neither profile nor PoR evidence requires successor-side possession.

Monotone attenuation along a chain is not itself new; capability tokens and recent delegation-chain proposals for AI agents already provide it (Section 17). The novelty claimed here is monotone attenuation of an *executor-relational admissibility space*, composed with authority attenuation and causal continuity. Section 16 maps a restricted subset of the premises onto an open-source implementation and states the gaps explicitly.

The result does not claim that PoP is useless, nor that deployed possession-based authorization systems do not authorize. Rather, it separates two roles that are often collapsed: *evidence that an actor controls something* and *the semantic condition that makes a continuation authorized*. In the model developed here, possession may help establish the former. Authentication fixes executor/profile facts; PoR establishes causal continuation; and C_i together with ρ_i bounds the authorization envelope.

As an intuition rather than a definition, PoP answers what artifact or key an entity demonstrates control over; PoR answers what execution occurrence is being continued; and authorization answers what that occurrence may exercise here, in its causal history. PoR is not by itself the authorization decision, and a PoP transcript is not by itself a relational profile or a causal history.

The picture is one of two layers. At admission, a sound evidence mechanism establishes facts about the executor and fixes its profile $q_J(x)$. PoP may provide holder-binding evidence only when combined with a trusted binding from the possessed artifact to the executor and the asserted classes; the evidence mechanism need not use successor-side PoP. Once the profile is established, PoR determines whether the occurrence is a causal continuation, C_i bounds which authority may continue, and ρ_i both tests and bounds the executor relationships through which it may continue. Treating possession as the ground of authority collapses the two layers. The collapse is harmless at a single hop, where capabilities work precisely because designation and authority coincide. It becomes visible across hops, because a lineage-invariant possession-based decision cannot distinguish occurrences that differ only in causal history (Remark 1).

2 Background and Previous Result

2.1 Authority as a causal continuation

Let O be a set of operations and R a set of resources. A privilege is

$$(o, r) \in O \times R.$$

A discrete execution is a causal chain

$$\pi = \langle (s_0, C_0), (s_1, C_1), \dots, (s_n, C_n) \rangle,$$

where s_i is an execution step and

$$C_i \subseteq O \times R$$

is its authority context.

The previous PoC model defines a single-hop Proof of Relationship by

$$\text{PoR}(s_i, s_{i+1}),$$

which holds when s_{i+1} is a valid causal continuation of s_i within the same execution lineage [1]. A transition is valid when it is both causally related and non-expansive:

$$\text{Valid}(s_i, C_i, s_{i+1}, C_{i+1}) \iff \text{PoR}(s_i, s_{i+1}) \wedge C_{i+1} \subseteq C_i.$$

The whole chain satisfies Proof-of-Continuity when every adjacent transition is valid:

$$\text{PoC}(\pi) \iff \bigwedge_{i=0}^{n-1} (\text{PoR}(s_i, s_{i+1}) \wedge C_{i+1} \subseteq C_i).$$

Proposition 1 (Origin-bounded authority, previous result). *If $\text{PoC}(\pi)$ holds, then*

$$C_n \subseteq C_{n-1} \subseteq \dots \subseteq C_0.$$

Consequently, if $(o, r) \notin C_0$, then

$$(o, r) \notin C_i$$

for every i .

Proof. Immediate by transitivity of set inclusion.

2.2 Confused deputy as authority/causality mismatch

The classical confused deputy occurs when a request causes a deputy to exercise authority that does not belong to the request's authority context. Capabilities address the classical single-invocation form by fusing designation and authority [8, 10]. In a distributed or multi-step execution, however, an executor may hold several unrelated authority sources simultaneously. The PIC project's N+1 Invalid State problem statement documents this authority-mixing motivation [5].

Let a request originate with authority context C_0 . If an executor later uses a privilege

$$(o, r) \notin C_0$$

because it independently possesses that privilege, then authority and causality have diverged. The previous PoC result rules this out as valid behavior because every valid descendant context is a subset of C_0 .

The present work preserves this result and adds a second closure dimension: a valid descendant may not expand the relational class of executions through which origin authority is permitted to continue.

3 Execution Steps and Executors

The previous model uses s_i for a formal execution step. Operationally, that step is realized by an executor such as a service, workload, process, function, tool, agent, or hardware component.

Definition 1 (Executor space). *Let*

$$S$$

be the set of execution steps and let

$$X$$

be the set of executor instances. Define

$$\text{exec} : S \rightarrow X.$$

For each $s_i \in S$, write

$$x_i := \text{exec}(s_i).$$

A causal chain

$$s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n$$

therefore induces an executor sequence

$$x_0, x_1, \dots, x_n.$$

The distinction between step and executor is essential. The same executor may realize several distinct occurrences:

$$s_i \neq s_j \quad \text{while} \quad x_i = x_j.$$

Conversely, a lineage may cross many different executors.

Authorization therefore cannot be reduced to executor identity alone. The relevant object is an *occurrence* of an executor in a causal execution and the relationships under which that occurrence is admitted.

4 Relational Equivalence and Executor Quotients

4.1 Equivalence relative to an authorization characteristic

We now formalize the idea that two physically different executors may be equivalent for one authorization purpose.

Definition 2 (Authorization-relevant equivalence). *Let $\sim_j$ be an equivalence relation on X :*

$$\sim_j \subseteq X \times X.$$

For $x, y \in X$,

$$x \sim_j y$$

means that x and y are indistinguishable with respect to relational characteristic j .

Since $\sim_j$ is an equivalence relation, it is reflexive, symmetric, and transitive. It partitions X into disjoint equivalence classes.

Definition 3 (Equivalence class). *For $x \in X$,*

$$[x]_{\sim_j} := \{y \in X : y \sim_j x\}.$$

The class contains every executor that is indistinguishable from x under the selected relation.

Definition 4 (Relational quotient). *The quotient induced by $\sim_j$ is*

$$\mathcal{R}_j := X / \sim_j$$

with

$$\mathcal{R}_j = \{[x]_{\sim_j} : x \in X\}.$$

We call $\mathcal{R}_j$ a relational quotient.

The quotient operation intentionally discards distinctions that the relation declares irrelevant. It does *not* assert that two concrete executors are identical.

4.2 A concrete quotient example

Let

$$X = \{a, b, c, d, e, f\}.$$

Suppose $\sim_1$ induces the partition

$$\{a, b\}, \quad \{c, d, e\}, \quad \{f\}.$$

Then

$$\mathcal{R}_1 = X / \sim_1 = \{\{a, b\}, \{c, d, e\}, \{f\}\}.$$

Since

$$a \sim_1 b,$$

we have

$$[a]_{\sim_1} = [b]_{\sim_1} = \{a, b\}.$$

Thus the quotient identifies a and b only with respect to the characteristic represented by $\sim_1$.

Now let a second relation $\sim_2$ induce

$$\{a, c, f\}, \quad \{b, d\}, \quad \{e\}.$$

Then

$$\mathcal{R}_2 = X / \sim_2 = \{\{a, c, f\}, \{b, d\}, \{e\}\}.$$

The same executor a therefore has

$$[a]_{\sim_1} = \{a, b\}$$

but

$$[a]_{\sim_2} = \{a, c, f\}.$$

There is no contradiction. The same executor is being observed through two different authorization-relevant relations.

4.3 Quotient maps

Each equivalence relation has a canonical quotient map

$$q_j : X \rightarrow \mathcal{R}_j$$

defined by

$$q_j(x) = [x]_{\sim_j}.$$

The map says: retain only the relational class of x and forget its irrelevant concrete differences.

This gives the intended reading:

$$\begin{aligned} X / \sim_j = \\ \text{“}X \text{ viewed at the resolution} \\ \text{determined by } \sim_j \text{”}. \end{aligned}$$

5 Multiple Relational Views

A single executor may participate in several authorization-relevant relationships. Let J index a family

$$\{\sim_j\}_{j \in J}.$$

Each relation produces its own quotient:

$$\mathcal{R}_j = X / \sim_j.$$

Definition 5 (Relational profile map). *Define*

$$q_J : X \rightarrow \prod_{j \in J} \mathcal{R}_j$$

by

$$q_J(x) = ([x]_{\sim_j})_{j \in J}.$$

The realizable relational profile space is

$$\mathcal{Q}_J := q_J(X) \subseteq \prod_{j \in J} \mathcal{R}_j.$$

The use of the image $q_J(X)$, rather than the full product, records only combinations of quotient classes realized by some executor.

5.1 Combined equivalence

The family of relations also induces a combined equivalence:

$$x \equiv_J y \iff \forall j \in J, x \sim_j y.$$

Equivalently,

$$\equiv_J = \bigcap_{j \in J} \sim_j.$$

Two executors are therefore equivalent under J precisely when their entire relational profiles agree:

$$x \equiv_J y \iff q_J(x) = q_J(y).$$

Proposition 2 (Factorization through a quotient). *Let Y be a set, $\equiv$ an equivalence relation on Y , and $q : Y \rightarrow Y/\equiv$ the canonical quotient map. A policy*

$$A : Y \rightarrow \{0, 1\}$$

is invariant under $\equiv$, i.e.

$$y \equiv y' \implies A(y) = A(y'),$$

if and only if there exists

$$\bar{A} : Y/\equiv \rightarrow \{0, 1\}$$

such that

$$A = \bar{A} \circ q.$$

When it exists, $\bar{A}$ is unique.

Proof. If $A = \bar{A} \circ q$ and $y \equiv y'$, then $q(y) = q(y')$, hence $A(y) = A(y')$. Conversely, for a class $c \in Y/\equiv$ choose any y with $q(y) = c$ and define $\bar{A}(c) := A(y)$. Invariance makes this value independent of the chosen representative, and $A = \bar{A} \circ q$ by construction. Uniqueness follows because q is surjective.

For executors, take $Y = X$ and $\equiv = \equiv_J$. Since $x \equiv_J y$ iff $q_J(x) = q_J(y)$, the profile map induces a bijection $X/\equiv_J \cong \mathcal{Q}_J$. A policy on executors is therefore invariant under $\equiv_J$ iff it factors as

$$A = \bar{A} \circ q_J, \quad \bar{A} : \mathcal{Q}_J \rightarrow \{0, 1\}.$$

This is the formal statement that an authorization rule may operate on relational equivalence classes rather than on concrete executor identity. The proposition is the standard universal property of a quotient; its interest here lies in what it unifies.

Remark 1 (The previous result as a quotient instance). *In [1], an event is an occurrence $e = (o, r, \ell) \in E := O \times R \times \mathcal{L}_{\text{prev}}$, where $\mathcal{L}_{\text{prev}}$ is that paper's history space, and the projection $p : E \rightarrow O \times R$ forgets the lineage. Let $\sim_{(o,r)}$ be the kernel relation of p ,*

$$e \sim_{(o,r)} e' \iff p(e) = p(e'),$$

and $q : E \rightarrow E/\sim_{(o,r)}$ its canonical quotient map. Since p is surjective, there is a canonical bijection $\phi : E/\sim_{(o,r)} \rightarrow$

$O \times R$ with $p = \phi \circ q$. Up to this bijection, p is the quotient map, and a policy factors through p iff it factors through q . The possession-based policies defined there, which factor through p or equivalently are lineage-invariant (Definition 7 of [1]), are exactly the policies invariant under $\sim_{(o,r)}$. The result that lineage-invariant policies cannot individuate occurrences (Theorem 3 of [1]) is Proposition 2 applied to $Y = E$ and $\equiv = \sim_{(o,r)}$: two events in the same class receive the same verdict. The companion result (Theorem 4 of [1]) establishes that a policy separating two events collapsed by the lineage-forgetting projection must read finer, lineage-sensitive information. This identifies the need for a finer observation; it does not by itself establish the causal-continuity semantics developed here.

More generally, for any policy of the form $P = \bar{P} \circ \text{Obs}$,

$$\text{Obs}(s) = \text{Obs}(s') \implies P(s) = P(s').$$

Thus a lineage-invariant or artifact-local policy cannot distinguish two otherwise identical privilege uses solely by their different causal histories. A capability construction that verifies an equivalent lineage-sensitive continuation condition lies outside this invariant policy class; the factorization result does not claim that every capability system is lineage-invariant.

The present construction therefore does not add a structure beside that of the previous paper; it contains it. The lineage-forgetting projection is one relational quotient, and the executor relations $\sim_j$ are further ones, all governed by the same factorization property.

6 Relational Predicates

The quotient spaces classify executors. Authorization additionally needs a language for stating which relational profiles, and which transitions between profiles, are admissible.

The two layers have different jobs. An equivalence relation $\sim_j$ must be reflexive, symmetric and transitive, while relations such as delegation, acts-for, trust, parent and child, supervision, and causal precedence lack at least one property required of an equivalence relation, although some may satisfy one or more of them. The quotients are therefore *classification spaces*, not the authorization relation itself. The authorization-relevant relation between a predecessor and its successor is a predicate on $\mathcal{Q}_J \times \mathcal{Q}_J$, which may be directional and need not be an equivalence. A supervisory relation, for instance, is an atom $\text{Sup}(u, v)$ that holds when the class of u supervises the class of v .

6.1 Transition profile space

A causal hop involves two executors:

$$x_i = \text{exec}(s_i), \quad x_{i+1} = \text{exec}(s_{i+1}).$$

Define the transition profile space

$$\mathcal{T}_J := \mathcal{Q}_J \times \mathcal{Q}_J.$$

The relational observation of a concrete hop is

$$\tau_i := (q_J(x_i), q_J(x_{i+1})) \in \mathcal{T}_J.$$

6.2 Atomic and composite predicates

Definition 6 (Relational predicate). *A relational predicate is a function*

$$\rho : \mathcal{T}_J \rightarrow \{0, 1\}.$$

Its semantic extension is

$$\llbracket \rho \rrbracket := \{\tau \in \mathcal{T}_J : \rho(\tau) = 1\}.$$

Let

$$R_1, R_2, \dots$$

be atomic predicates over $\mathcal{T}_J$. We use the positive relational language generated by

$$\rho ::= R_k \mid (\rho \wedge \rho) \mid (\rho \vee \rho).$$

We write $\text{Pred}(\mathcal{T}_J)$ for the set of positive relational predicates generated by this grammar from the atoms. Relational envelopes range over $\text{Pred}(\mathcal{T}_J)$, not over arbitrary functions $\mathcal{T}_J \rightarrow \{0, 1\}$.

Why no negation. The answer depends on what a boundary knows about an atom. Each atom is a fixed subset of $\mathcal{T}_J$, and the profiles in τ are verified. Some atoms are decidable from τ itself, such as equality or inequality of two profile components. For those, the atom and its negation are equally decidable, and nothing is lost by admitting $\neg$. The separation-of-duty atom of Section 18.3 is of this kind, which is why it may contain $\neq$ and $\neq$. For other atoms the extension is not known to the boundary: a supervision relation that depends on an external organizational chart, or a relation recorded in a delegation registry. The boundary then learns membership only through evidence, and the truth of the atom must be kept apart from the availability of evidence for it.

Let E be the evidence available to a boundary, and write $E \vdash R_k(\tau)$ when E supports $R_k(\tau)$. The relation lifts to positive predicates:

$$E \vdash (\rho \wedge \sigma)(\tau) \iff E \vdash \rho(\tau) \text{ and } E \vdash \sigma(\tau),$$

$$E \vdash (\rho \vee \sigma)(\tau) \iff E \vdash \rho(\tau) \text{ or } E \vdash \sigma(\tau).$$

We assume two properties of atoms. Evidence is *sound* when $E \vdash R_k(\tau)$ implies $R_k(\tau)$. It is *monotone* when $E \subseteq E'$ and $E \vdash R_k(\tau)$ imply $E' \vdash R_k(\tau)$. Both properties lift by induction to every positive predicate. By soundness, a boundary that admits on $E \vdash \rho_i(\tau_i)$ admits only transitions that satisfy ρ_i . By monotonicity, withholding evidence cannot produce an admission that full evidence would deny. Negation breaks this for atoms of the second kind: supporting $\neg R_k(\tau)$ requires evidence of

absence, which an open world does not provide, and reading “not supported” as “false” would let missing evidence cause admission.

The positive fragment is therefore the conservative choice, safe for both kinds of atom. Negation is not excluded on mathematical grounds. With $\neg$ added and formulas still identified extensionally, semantic implication and the attenuation arguments can be retained, and the generated semantic algebra is Boolean. The positive-DNF containment criterion of Lemma 1, however, does not extend unchanged to formulas with negation. A deployment may admit $\neg$ on atoms decidable at the boundary, or under a closed-world assumption, but must use an appropriate implication check. The validity conditions below are stated at the level of truth; a boundary discharges them through sound evidence.

Conjunction and disjunction have their ordinary set semantics:

$$\llbracket \rho \wedge \sigma \rrbracket = \llbracket \rho \rrbracket \cap \llbracket \sigma \rrbracket,$$

and

$$\llbracket \rho \vee \sigma \rrbracket = \llbracket \rho \rrbracket \cup \llbracket \sigma \rrbracket.$$

Thus one causal execution may require, for example,

$$R_1 \wedge R_2,$$

another may allow

$$R_1 \vee R_2,$$

and a later continuation may refine the condition to

$$(R_1 \vee R_2) \wedge R_3.$$

7 The Relational Predicate Order

We now define the order that will carry relational continuity.

Definition 7 (Relational refinement). *For relational predicates ρ' and ρ , define*

$$\rho' \preceq_R \rho$$

iff

$$\rho' \Rightarrow \rho,$$

that is, $\rho'(\tau) = 1$ implies $\rho(\tau) = 1$ for every $\tau \in \mathcal{T}_J$. Equivalently,

$$\llbracket \rho' \rrbracket \subseteq \llbracket \rho \rrbracket.$$

Throughout, $\Rightarrow$ between predicates denotes this extensional implication, under the fixed interpretation of the atoms.

A stronger predicate therefore denotes a smaller set of admissible transitions.

For example,

$$R_1 \Rightarrow R_1 \vee R_2,$$

so

$$R_1 \preceq_R R_1 \vee R_2.$$

Likewise,

$$R_1 \wedge R_3 \Rightarrow R_1,$$

so

$$R_1 \wedge R_3 \preceq_R R_1.$$

Hence the sequence

$$R_1 \vee R_2 \succeq_R R_1 \succeq_R R_1 \wedge R_3$$

is monotonically restrictive.

Proposition 3 (Relational predicate lattice). *Identify positive relational predicates over $\mathcal{T}_J$ that have the same semantic extension. Ordered by $\preceq_R$, the resulting classes form a distributive lattice with meet $\wedge$ and join $\vee$.*

Proof. Under the semantic map $\rho \mapsto \llbracket \rho \rrbracket$, conjunction maps to set intersection and disjunction maps to set union. The extensions of positive predicates are therefore closed under $\cap$ and $\cup$, and form a sublattice of the powerset of $\mathcal{T}_J$ ordered by inclusion. Every sublattice of a distributive lattice is distributive.

A second identification is possible: the free positive algebra, in which formulas are identified when they are equivalent under *every* interpretation of the atoms. It is finer than the extensional one, because two atoms may have the same extension without being propositionally equivalent. The closure results below use only the extensional order. The syntactic check that follows is the interpretation-independent one.

This lattice is the formal version of a relational authorization space. A successor may move downward by refinement, thereby accepting fewer relational transitions, but may not move upward without an explicit re-authorization or a new origin.

Checking refinement. As a semantic condition, $\rho' \Rightarrow \rho$ depends on the extensions of the atoms. A verifier need not compute them, because an interpretation-independent refinement can be checked syntactically. Every formula in the positive grammar is finite and can be written in disjunctive normal form. Regard each conjunction as the finite set of atoms occurring in it.

Lemma 1 (Positive-DNF implication criterion). *Let*

$$\rho' = \bigvee_{a \in A} \bigwedge_{p \in P_a} p, \quad \rho = \bigvee_{b \in B} \bigwedge_{p \in Q_b} p$$

be positive formulas in DNF. Then $\rho' \Rightarrow \rho$ holds under every Boolean interpretation of the atoms iff

$$\forall a \in A \exists b \in B : Q_b \subseteq P_a.$$

Equivalently, every disjunct of ρ' contains all atoms of some disjunct of ρ .

Proof. If $Q_b \subseteq P_a$, every valuation satisfying the conjunction indexed by P_a also satisfies the one indexed by Q_b . The displayed condition therefore makes every disjunct of ρ' imply ρ , and hence makes $\rho' \Rightarrow \rho$ valid under every interpretation.

Conversely, suppose that for some $a \in A$, no Q_b is contained in P_a . Assign true exactly to the atoms in P_a and false to every other atom occurring in either formula. This valuation satisfies the a -th disjunct of ρ' . For every $b \in B$, choose $p_b \in Q_b \setminus P_a$; that atom is false, so no disjunct of ρ is satisfied. Thus $\rho' \not\Rightarrow \rho$, proving the contrapositive.

In particular, conjoining a further predicate ($\rho' = \rho \wedge \sigma$) and deleting disjuncts are always refinements. The criterion of Lemma 1 is sound and decidable from the formulas alone. It is conservative for the fixed semantic interpretation used by the model: particular atom extensions may make $\rho' \Rightarrow \rho$ hold even when the interpretation-independent criterion fails.

8 Relational Continuity Along a Causal Chain

8.1 Relational envelopes

Associate with every execution step s_i both:

1. an authority context

$$C_i \subseteq O \times R,$$

and

2. a relational envelope

$$\rho_i \in \text{Pred}(\mathcal{T}_J).$$

The relational envelope specifies the class of executor-profile transitions that remain admissible at that point in the lineage.

Definition 8 (Authorization-aware causal history). *Let $\mathcal{L}$ be the set of finite nonempty sequences*

$$\widehat{\ell}_i := \langle (s_0, C_0, \rho_0), \dots, (s_i, C_i, \rho_i) \rangle,$$

where $s_k \in S$, $C_k \subseteq O \times R$, and $\rho_k \in \text{Pred}(\mathcal{T}_J)$. These histories are authorization-aware: their endpoint contains the state carried at that occurrence. Write $\widehat{\ell}_i \preceq_L \widehat{\ell}_j$ when the former sequence is a prefix of the latter. The index records a position inside this particular history; it is not a global clock and does not identify an occurrence without the history.

8.2 Extended valid transition

Definition 9 (Relationally valid transition). *Let*

$$x_i = \text{exec}(s_i), \quad x_{i+1} = \text{exec}(s_{i+1}),$$

and

$$\tau_i = (q_J(x_i), q_J(x_{i+1})).$$

The transition from

$$(s_i, C_i, \rho_i)$$

to

$$(s_{i+1}, C_{i+1}, \rho_{i+1})$$

is relationally valid iff

$$\begin{aligned} \text{Valid}_R(s_i, C_i, \rho_i, s_{i+1}, C_{i+1}, \rho_{i+1}) &\iff \text{PoR}(s_i, s_{i+1}) \\ &\quad \wedge C_{i+1} \subseteq C_i \\ &\quad \wedge \rho_i(\tau_i) \\ &\quad \wedge (\rho_{i+1} \Rightarrow \rho_i). \end{aligned}$$

The four conjuncts have different roles:

1. $\text{PoR}(s_i, s_{i+1})$ establishes causal continuation;
2. $C_{i+1} \subseteq C_i$ prevents authority expansion;
3. $\rho_i(\tau_i)$ requires the concrete executor transition to satisfy the currently admitted relation; and
4. $\rho_{i+1} \Rightarrow \rho_i$ prevents relational expansion.

Definition 10 (PIC-compatible realization). *A concrete authorization system is PIC-compatible with respect to an authorization-aware lineage model when every admitted non-origin transition $s_i \rightarrow s_{i+1}$ is represented by endpoint states (s_i, C_i, ρ_i) and $(s_{i+1}, C_{i+1}, \rho_{i+1})$, indexed by the corresponding causal-history prefixes, and the system:*

1. *soundly establishes* $\text{PoR}(s_i, s_{i+1})$;
2. *enforces* $C_{i+1} \subseteq C_i$;
3. *soundly establishes the profile facts used in* $\tau_i = (q_J(x_i), q_J(x_{i+1}))$ *and checks* $\rho_i(\tau_i)$; *and*
4. *enforces* $\rho_{i+1} \Rightarrow \rho_i$.

Thus the admitted transitions realize Definition 9, independently of which artifacts or policy engine carry and check its premises. This definition classifies transition admission only. It does not imply that every real-world effect is mediated or attributed to the correct lineage; those are the separate premises of Assumptions 1 and 2.

Two design points concern the third conjunct. First, hop i is governed by the predecessor's envelope ρ_i , not by the successor's ρ_{i+1} . The envelope carried at s_i constrains the transition that continues s_i ; the envelope ρ_{i+1} constrains the transitions that will continue s_{i+1} . Requiring $\rho_{i+1}(\tau_i)$ as well would impose a stronger, retroactive policy that the model does not need.

Second, τ_i refers to the class of x_{i+1} , an executor that may not exist, or may not yet be selected, when s_i emits its continuation. The condition $\rho_i(\tau_i)$ is therefore evaluated by the receiving side at acceptance time, not by the predecessor at emission time. This is the temporal gap that Proof-of-Continuity addresses for authority [1], applied to the envelope: the predecessor fixes the envelope, and any successor that later appears is admitted only if it falls inside it. The PIC project documents the corresponding engineering motivation as the N+1 unknown-executor problem [3].

PoR supplies the causal spine. The authority and relational contexts are the two monotone payloads propagated along that spine.

9 Dual Monotonicity

Lemma 2 (Dual Monotonicity). *Let*

$$\pi = \langle (s_0, C_0, \rho_0), \dots, (s_n, C_n, \rho_n) \rangle$$

be a chain in which every adjacent transition is relationally valid. Then

$$C_n \subseteq C_{n-1} \subseteq \dots \subseteq C_0$$

and

$$\rho_n \preceq_R \rho_{n-1} \preceq_R \dots \preceq_R \rho_0.$$

In particular,

$$C_n \subseteq C_0$$

and

$$\rho_n \Rightarrow \rho_0.$$

Proof. Every valid transition satisfies

$$C_{i+1} \subseteq C_i.$$

Transitivity of set inclusion gives

$$C_n \subseteq C_0.$$

Every valid transition also satisfies

$$\rho_{i+1} \Rightarrow \rho_i.$$

Transitivity of logical implication gives

$$\rho_n \Rightarrow \rho_0.$$

The two monotonicities decrease in different mathematical spaces:

$$C_i$$

attenuates under set inclusion, while

$$\rho_i$$

attenuates under logical implication.

They nevertheless express the same continuity principle:

A valid successor may preserve or restrict what its predecessor allowed, but it may not expand it.

The proof is short by design. Validity imposes monotonicity at each hop, and the lemma is its transitive closure, so the result is close to its definition. The substance of the model lies elsewhere: in the choice of the two ordered spaces, in their composition with PoR along one lineage, in the reading of the earlier projection result as a quotient instance, and in the witness construction of Section 14.4, which conditionally realizes the premises under its explicit assumptions.

10 Relational Closure

Dual monotonicity implies more than endpoint restriction. Every concrete transition in the lineage remains inside the relational envelope established at the origin.

Theorem 1 (Origin Relational Closure). *Let*

$$\tau_i = (q_J(x_i), q_J(x_{i+1}))$$

be the relational observation of hop i . If every adjacent transition is relationally valid, then

$$\tau_i \in \llbracket \rho_0 \rrbracket$$

for every $i = 0, \dots, n-1$.

Proof. Validity gives

$$\rho_i(\tau_i) = 1,$$

hence

$$\tau_i \in \llbracket \rho_i \rrbracket.$$

By Dual Monotonicity,

$$\rho_i \Rightarrow \rho_0,$$

so

$$\llbracket \rho_i \rrbracket \subseteq \llbracket \rho_0 \rrbracket.$$

Therefore

$$\tau_i \in \llbracket \rho_0 \rrbracket.$$

Thus a valid descendant cannot introduce an executor relation that the origin relational envelope excluded.

Corollary 1 (No relational expansion). *If*

$$\tau \notin \llbracket \rho_0 \rrbracket,$$

then τ cannot occur as a valid transition anywhere in the same relationally continuous lineage.

11 The Product Authorization State

The two monotone dimensions can be represented as one ordered state.

Definition 11 (Relational authorization state). *For the history of Definition 8, define*

$$\Gamma(\widehat{\ell}_i) := (C_i, [\rho_i]_{\text{sem}}),$$

and abbreviate it by Γ_i . This is well-defined because the endpoint tuple of ℓ_i contains C_i and ρ_i . The predicate coordinate is its semantic-equivalence class from Proposition 3; replacing ρ_i by a predicate with the same extension does not change the state.

Definition 12 (Product attenuation order). *For states*

$$\Gamma' = (C', \rho') \quad \text{and} \quad \Gamma = (C, \rho),$$

define

$$\Gamma' \preceq \Gamma$$

iff

$$C' \subseteq C \quad \text{and} \quad \rho' \Rightarrow \rho.$$

Proposition 4 (The product order is a partial order). *On*

$$\mathcal{P}(O \times R) \times (\text{Pred}(\mathcal{T}_J)/\equiv_{\text{sem}}),$$

where $\rho \equiv_{\text{sem}} \sigma$ iff $\llbracket \rho \rrbracket = \llbracket \sigma \rrbracket$, the relation $\preceq$ is a partial order. If predicates are retained as syntactic formulas instead, the same definition is only a preorder.

Proof. Reflexivity and transitivity follow componentwise from set inclusion and logical implication. If both $(C', \rho') \preceq (C, \rho)$ and $(C, \rho) \preceq (C', \rho')$, then $C' = C$ and $\llbracket \rho' \rrbracket = \llbracket \rho \rrbracket$; hence the two states are equal in the displayed quotient. Without quotienting formulas by semantic equivalence, distinct but equivalent formulas witness the failure of antisymmetry.

Every valid transition therefore satisfies

$$\Gamma_{i+1} \preceq \Gamma_i.$$

Hence:

$$\Gamma_n \preceq \Gamma_{n-1} \preceq \dots \preceq \Gamma_0.$$

Lemma 3 (Combined Closure). *For every valid descendant state $\Gamma_i = (C_i, [\rho_i]_{\text{sem}})$,*

$$C_i \subseteq C_0$$

and

$$\llbracket \rho_i \rrbracket \subseteq \llbracket \rho_0 \rrbracket.$$

Proof. Immediate from Dual Monotonicity and the semantic equivalence

$$\rho_i \Rightarrow \rho_0 \iff \llbracket \rho_i \rrbracket \subseteq \llbracket \rho_0 \rrbracket.$$

The origin therefore bounds both:

which privileges may continue

and

through which executor relationships
they may continue

12 Causal History Grows While Authorization Freedom Shrinks

The lineage itself is not one of the shrinking objects. Under the prefix order of Definition 8,

$$\widehat{\ell}_0 \preceq_L \widehat{\ell}_1 \preceq_L \cdots \preceq_L \widehat{\ell}_n.$$

The temporal axis here is causal prefix order, not wall-clock time: timestamps, expiry and elapsed time may be evidence or policy inputs, but none determines the position $\widehat{\ell}_i$ in the causal execution history.

Thus the three dimensions move together but not identically:

$$\begin{array}{ccccccc} s_0 & \rightarrow & s_1 & \rightarrow & s_2 & \rightarrow & \cdots s_n \\ C_0 & \supseteq & C_1 & \supseteq & C_2 & \supseteq & \cdots C_n \\ \rho_0 & \Leftarrow & \rho_1 & \Leftarrow & \rho_2 & \Leftarrow & \cdots \rho_n \\ \widehat{\ell}_0 & \preceq_L & \widehat{\ell}_1 & \preceq_L & \widehat{\ell}_2 & \preceq_L & \cdots \widehat{\ell}_n \end{array}$$

where

$$\rho_{i+1} \Rightarrow \rho_i.$$

More generally, if $\widehat{\ell}_i \preceq_L \widehat{\ell}_j$ and every intervening transition is relationally valid, Dual Monotonicity gives

$$\Gamma(\widehat{\ell}_j) \preceq \Gamma(\widehat{\ell}_i).$$

The precise statement is therefore not that a lineage in $\mathcal{L}$ itself decreases. Rather, causal history grows by prefix extension while the authorization contexts *carried along that history* attenuate in two dimensions.

This yields the structural principle:

causal evidence accumulates while
authorization freedom attenuates.

13 The Confused Deputy Under Dual Continuity

13.1 Authority mixing

Suppose an executor D processes a request whose origin context is C_0 while simultaneously holding an independent authority source C_D . Let

$$(o, r) \notin C_0 \quad \text{but} \quad (o, r) \in C_D.$$

A lineage-invariant possession model may authorize the action by selecting C_D , even though the request itself did not carry that authority. This is the authority/causality mismatch that produces the confused deputy in the multi-step setting [1].

13.2 Valid exercise

The authorization rule of [1] links the exercise of a privilege to the context carried at the step that exercises it. We restate it for the relational model.

Definition 13 (Valid exercise). *Let*

$$\pi = \langle (s_0, C_0, \rho_0), \dots, (s_n, C_n, \rho_n) \rangle$$

be a chain. A step s_i of π validly exercises (o, r) iff

$$(o, r) \in C_i$$

and either $i = 0$, or the transition from s_{i-1} to s_i is relationally valid; in the latter case $\rho_{i-1}(\tau_{i-1}) = 1$ in particular.

The relational condition is indexed by the hop that enters s_i , because an envelope governs the transition that continues the step carrying it.

Proposition 5 (No in-lineage exercise beyond the origin). *If every transition of π up to s_i is relationally valid and s_i validly exercises (o, r) , then*

$$(o, r) \in C_0.$$

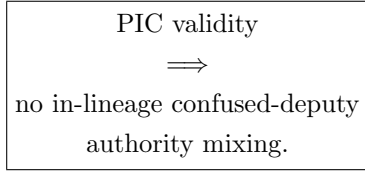
Proof. By Definition 13, $(o, r) \in C_i$. The prefix $\langle (s_0, C_0, \rho_0), \dots, (s_i, C_i, \rho_i) \rangle$ is itself a chain whose adjacent transitions are all relationally valid, so Combined Closure applies to it and gives $C_i \subseteq C_0$. Hence $(o, r) \in C_0$.

Interpretation: no in-lineage authority mixing. Even if $\text{exec}(s_i) = D$ and D independently holds authority for (o, r) from a source outside the lineage, that fact does not inject (o, r) into C_i . The model neither identifies C_i with all authority held by $\text{exec}(s_i)$ nor supplies an update rule that adds independently held privileges to C_{i-1} . Such an update would be an authority expansion unless the added privileges were already in C_{i-1} , whereas validity requires $C_i \subseteq C_{i-1}$. Thus independent authority may exist while remaining semantically external to the current lineage:

$$\begin{array}{c} (o, r) \notin C_0 \\ \implies \\ (o, r) \text{ cannot be a valid} \\ \text{in-lineage exercise.} \end{array}$$

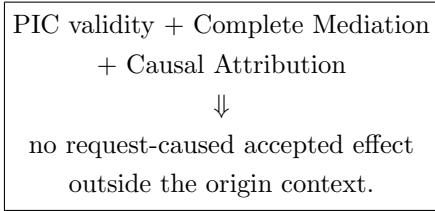
This conclusion is independent of executor identity. Different steps may be realized by different executors, but authority follows the execution lineage, not the union of their ambient privilege sets.

For example, let $C_0(req) = \{(read, A)\}$, while deputy D independently holds $(write, B)$. Even if $exec(s_i) = D$, validity gives $C_i \subseteq C_0(req)$, hence $(write, B) \notin C_i$. The deputy possesses the authority; the execution does not. This is the precise sense in which



The result is internal to the PIC semantics and uses neither $Priv(exec(s_i))$, Complete Mediation, nor Causal Attribution.

Proposition 5, however, only constrains steps of the lineage. It does not by itself establish that every real-world effect caused by a request is presented in that lineage. In particular, a deputy may perform an action outside it by opening a lineage of its own, rooted in its own privileges on the target the request designated. This is Hardy's compiler, which writes the billing file with its own authority as if it were housekeeping [8]. The previous paper identifies this gap as lineage origination and leaves it to future work [1]. Closing it requires the additional premises stated below. In compact form, the global result has the stronger premise:



13.3 Exclusion of the confused deputy

We first make the objects used by the result explicit. Let Req be the set of requests and Eff the set of effects that occur. Every request has an authorization-aware lineage through a map

$$ReqLineage : Req \rightarrow \mathcal{L}, \quad req \mapsto \hat{\ell}_{req},$$

and $C_0(req)$ is the authority context in the first tuple of $\hat{\ell}_{req}$. Let

$$step : Eff \rightarrow S, \quad realize : Eff \rightarrow O \times R,$$

associate an effect with the step that performs it and the privilege it realizes. Let

$$Lineage : Eff \rightarrow \mathcal{L}$$

be the lineage to which the enforcement system attributes the effect, and let

$$CausedBy \subseteq Eff \times Req$$

be the semantic causal relation between effects and requests. Finally, write $IndependentAuthority(D, (o, r))$ when executor D holds authority for (o, r) from a source other than the request context. An effect e is performed by D when $exec(step(e)) = D$.

The distinction between $CausedBy$ and $Lineage$ is intentional. The former is the causal fact that the security statement concerns; the latter is the attribution made available to enforcement.

Definition 14 (Confused deputy, lineage form). *A confused deputy occurs iff there exist $req \in Req$, $e \in Eff$, $D \in X$, and $(o, r) \in O \times R$ such that*

$$\begin{aligned} CausedBy(e, req), & \quad realize(e) = (o, r), \\ exec(step(e)) = D, & \quad (o, r) \notin C_0(req), \\ IndependentAuthority(D, (o, r)). \end{aligned}$$

The previous paper states its privilege-exclusion condition as $(o, r) \notin Priv(U)$ for the originating principal U . If the origin gate selects $C_0(req) \subseteq Priv(U)$, then that condition implies $(o, r) \notin C_0(req)$. Under this explicit origin-selection premise, every confused deputy in the sense of [1] is therefore one in the sense of Definition 14. This comparison is not used by the theorem below, whose abstract initial context is simply $C_0(req)$. The lineage form is the stronger target when the premise holds: it also covers privileges that the originating principal holds but deliberately left out of $C_0(req)$.

Definition 15 (Accepted effect). *For $\hat{\ell} \in \mathcal{L}$, an effect e is accepted in $\hat{\ell}$ for (o, r) , written $Accepted(e, \hat{\ell}, (o, r))$, iff $realize(e) = (o, r)$, $step(e) = s_k$ for some tuple (s_k, C_k, ρ_k) of $\hat{\ell}$, every transition in the prefix ending at that tuple is relationally valid, and s_k validly exercises (o, r) .*

Here $realize(e)$ denotes the operation and actual resource on which the effect occurs after name resolution, aliasing, redirection and other designation processing. Checking only an input string, pathname or logical designation is insufficient if it may resolve to a different resource. Complete mediation requires the verifier to authorize the realized pair (o, r) . Predecessor binding and destination binding are therefore separate: the checkpoint binds a continuation to its predecessor and carried state, while the complete mediator binds authorization to the operation and resource actually realized. The checkpoint alone does not make an unresolved designation an authorized destination.

Assumption 1 (Complete mediation). *Every occurring effect is accepted under its attributed lineage:*

$$\forall e \in Eff, \quad Accepted(e, Lineage(e), realize(e)).$$

No path to the affected resource bypasses the verifier.

Assumption 2 (Causal attribution). *Every effect causally triggered by a request is presented as a step of*

that request's lineage:

$$\begin{aligned} \forall e \in \text{Eff} \ \forall req \in \text{Req}, \\ \text{CausedBy}(e, req) \implies \text{Lineage}(e) = \widehat{\ell}_{req}. \end{aligned}$$

A third property is related but distinct.

Assumption 3 (Origination control). *A new lineage is opened only at an origin gate independent of every executor, and executors hold no authority of their own outside lineages opened there.*

Origination control and causal attribution are not equivalent. Origination control restricts *who* may open a lineage, but the gate may still be invoked because of a request, and would then re-root a request-caused effect in a new lineage: origination control holds while attribution fails. Conversely, attribution can hold while executors open genuinely independent lineages of their own. The theorem below uses complete mediation and causal attribution; origination control is one ingredient of how an architecture realizes them.

Theorem 2 (No request-caused effect beyond the origin). *Under Assumptions 1 and 2, with the authority-attenuation clause $C_{i+1} \subseteq C_i$ built into relational validity, for every $req \in \text{Req}$, $e \in \text{Eff}$, and $(o, r) \in O \times R$,*

$$\begin{aligned} \text{CausedBy}(e, req) \wedge \text{realize}(e) = (o, r) \\ \implies (o, r) \in C_0(req). \end{aligned}$$

Proof. Suppose an effect e caused by req realizes (o, r) . By Assumption 2, e is presented as a step s_k of $\widehat{\ell}_{req}$. By Assumption 1, $\text{Accepted}(e, \widehat{\ell}_{req}, (o, r))$ holds, so every transition of $\widehat{\ell}_{req}$ up to s_k is relationally valid and s_k validly exercises (o, r) . Applying Proposition 5 to the prefix of $\widehat{\ell}_{req}$ ending at s_k gives $(o, r) \in C_0$. Here $C_0 = C_0(req)$, the origin context of $\widehat{\ell}_{req}$.

Corollary 2 (Exclusion of the confused deputy). *Under Assumptions 1 and 2, the confused-deputy condition of Definition 14 is unsatisfiable. A fortiori, the condition of [1] is unsatisfiable as well.*

Proof. Theorem 2 contradicts $(o, r) \notin C_0(req)$. The independent-authority conjunct is not needed; the theorem proves the stronger statement that *no* request-caused effect, whether or not its executor independently holds the privilege, can exercise authority outside the request's origin context.

The contradiction is direct, and the enforcement properties that the proof does not derive sit in two named assumptions. The monotone-authority premise enters through relational validity and Proposition 5; the relational-envelope premise is not needed for the authority conclusion. The result does not say that a deputy cannot contain a logic error or attempt an operation outside $C_0(req)$. It says that, under complete mediation and

causal attribution, such an error cannot produce an *accepted* request-caused effect exercising authority outside that context. The deputy may possess the independent authority; possession does not make that authority admissible in the request's causal history.

What the construction realizes, and what it does not. The boundary-witnessed construction of Section 14.4 can discharge complete mediation for resources whose every access is forced through its boundaries; exclusive resource mediation is a deployment property, not a consequence of the checkpoint syntax. Under its Witness Assumptions, the construction discharges causal attribution for *explicit mediated handoffs*: a checkpoint is issued only for a step observed consuming the continuation of a verified predecessor. For resources subject to that exclusive mediation, origination control also prevents an executor from opening an independently privileged lineage outside the gate.

It does not realize attribution for causality that flows through state. A request may modify an executor's memory, a queue, a database, a cache, a file or shared memory; the request may then complete, and a later timer, asynchronous callback or background worker may read that state and obtain a new lineage at the origin gate. The resulting effect depends causally on the request without consuming its continuation, and no handoff observation will attribute it. Designation that flows as data is the typical case: a deputy performs housekeeping on a target it read from the request. Indirect signaling, covert channels and side channels can likewise influence control or data flow without appearing as an explicit mediated handoff. Information-flow tracking or a stronger execution model can address modeled overt channels, but ordinary tracking does not by itself establish attribution for every covert or side channel. Otherwise a further causal-attribution assumption is required, for example that executors retain no state across requests. The attribution itself therefore belongs to the trusted computing base of the enforcement architecture. Corollary 2 follows deductively from its two premises; the construction discharges attribution for mediated handoffs, and not beyond.

The canonical Hardy case falls precisely within this limitation when a request-supplied designation influences an effect through the deputy's independent authority without appearing as an explicit mediated handoff. The effect is semantically request-caused, but the witness construction alone may not attribute it to the request history. Exclusion of that case therefore remains conditional on Causal Attribution and Complete Mediation; this is a limit of the construction, not of the conditional theorem. PIC eliminates the authority-mixing step once an effect is inside the request's valid lineage. The remaining deployment problem is ensuring that request-caused effects cannot escape attribution to that lineage.

Exclusive mediated ingress. A strongly confined deployment may impose the sufficient operational condition that every modeled security-relevant external input capable of influencing a request-dependent effect enters as a boundary-mediated continuation associated with a lineage. Combined with complete mediation of effects and lineage-preserving treatment of every modeled internal channel, this gives the verifier an ingress and egress point for all such influence. This is a deployment condition, not a premise of the core closure results, and it does not establish attribution for unmodeled covert or side channels.

The relational component is not needed for this result. Its contribution is orthogonal: it closes the class of admissible executor relationships at the same time.

Corollary 3 (Dual closure against authority and relationship import). *In a relationally continuous lineage:*

1. *no step validly exercises, and no state carries, a privilege absent from C_0 ; and*
2. *no hop has a transition profile outside $\llbracket \rho_0 \rrbracket$, and no envelope carried along the lineage admits one.*

This is the strengthened closure result. Authority cannot be imported from an unrelated source, and relational admissibility cannot be widened in order to make an otherwise inadmissible continuation appear valid.

14 Authorization Without Successor-Side Proof of Possession

We now separate the semantic authorization rule from the mechanisms used to produce evidence for that rule.

14.1 Authentication and authorization are separate layers

Fixing the steps also fixes their actual executors, since $x_i = \text{exec}(s_i)$. To isolate the executor-dependent input, define the following counterfactual decision template for fixed $s_i, C_i, \rho_i, s_{i+1}, C_{i+1}, \rho_{i+1}$: $V : X \times X \rightarrow \{0, 1\}$ by

$$\begin{aligned} V(x, x') := & \text{PoR}(s_i, s_{i+1}) \wedge C_{i+1} \subseteq C_i \\ & \wedge \rho_i(q_J(x), q_J(x')) \\ & \wedge (\rho_{i+1} \Rightarrow \rho_i), \end{aligned}$$

so that

$$\begin{aligned} \text{Valid}_R(s_i, C_i, \rho_i, \\ s_{i+1}, C_{i+1}, \rho_{i+1}) \\ = V(x_i, x_{i+1}). \end{aligned}$$

PoR is a condition on the fixed steps, not on the two arguments of this template. Thus V is used only to expose factorization of the relational input; it does not assert that changing an executor while holding a step fixed produces another execution admitted by the model.

Proposition 6 (Relational executor dependence factors through the profile). *Conditional on fixed execution occurrences and authorization state, the executor-dependent relational component of the decision factors through $q_J \times q_J$: the function V is invariant under $\equiv_J$ in each argument, and therefore factors uniquely as*

$$V = \bar{V} \circ (q_J \times q_J), \quad \bar{V} : \mathcal{T}_J \rightarrow \{0, 1\}.$$

Proof. The executors occur in V only through $(q_J(x), q_J(x'))$. If $x \equiv_J y$ and $x' \equiv_J y'$, then $q_J(x) = q_J(y)$ and $q_J(x') = q_J(y')$, so $V(x, x') = V(y, y')$. Apply Proposition 2 to $Y = X \times X$ with the product equivalence $\equiv_J \times \equiv_J$. Its quotient is in canonical bijection with $\mathcal{Q}_J \times \mathcal{Q}_J = \mathcal{T}_J$ via $q_J \times q_J$, which is surjective onto $\mathcal{T}_J$.

The proposition is deliberately modest. It does not say that the executor is irrelevant to the decision: the occurrences determine their executors, and PoR depends on the occurrences. By itself it proves invariance under executor replacement once the profiles and the fixed occurrence-level data are given; it does not quantify over authentication mechanisms. We now state the corresponding, and still deliberately limited, mechanism claim explicitly.

For the fixed occurrence and authorization data used above, define

$$\begin{aligned} D(u, v) := & \text{PoR}(s_i, s_{i+1}) \wedge C_{i+1} \subseteq C_i \\ & \wedge \rho_i(u, v) \wedge (\rho_{i+1} \Rightarrow \rho_i), \\ & (u, v) \in \mathcal{T}_J. \end{aligned}$$

Definition 16 (Sound profile-evidence mechanism). *A profile-evidence mechanism M consists of an evidence space $\mathcal{E}_M$ and a partial extraction function*

$$\alpha_M : X \times \mathcal{E}_M \rightarrow \mathcal{Q}_J.$$

It is sound when

$$\alpha_M(x, e) = u \implies u = q_J(x)$$

for every input on which it is defined. This definition concerns only evidence for the executor profile; evidence for PoR and for external relational atoms remains subject to its own soundness assumptions.

Proposition 7 (Profile-preserving evidence-mechanism invariance). *Let M and N be profile-evidence mechanisms. If*

$$\begin{aligned} \alpha_M(x, e) = \alpha_N(y, f) = u, \\ \alpha_M(x', e') = \alpha_N(y', f') = v, \end{aligned}$$

then both mechanisms induce the same authorization verdict $D(u, v)$. In particular, two sound mechanisms defined for the same concrete executors x_i, x_{i+1} induce the same verdict.

Proof. The definition of D has no argument for a mechanism or for its evidence; its executor-dependent input is exactly (u, v) . Equality of those inputs therefore gives equality of verdicts. For sound mechanisms applied to the same executors, Definition 16 gives $u = q_J(x_i)$ and $v = q_J(x_{i+1})$ for each mechanism.

This proposition is an output-preservation result. It does not claim that all mechanisms can produce evidence, that they have equal security properties, or that the evidence mechanism for PoR may be replaced without re-establishing its soundness. It states precisely the separation used here: once a sound mechanism establishes the same profile facts, its internal use or non-use of PoP is not inspected by D . Possession, where it is used, therefore contributes evidence to the layer that establishes membership in a relational class. Holding the same causal and authorization state fixed, the accepted transition is the same; consequently, the closure results downstream of that transition are unchanged. This does not transfer the security properties of one evidence mechanism to another: each must separately satisfy its soundness obligation. Authentication establishes the executor profiles, PoR establishes causal continuation, C_i bounds the privileges that may continue, and ρ_i bounds the profile transitions through which they may continue.

Membership is established at admission. The profile is fixed when the boundary accepts the step and records it in the checkpoint of Section 14.4. “Once authenticated” therefore means for this admission, not forever; how class membership changes over time, and how revocation affects it, remain among the limitations of Section 19.

Section 16 describes a realization that takes this layered route: the successor authenticates with possession, and the decision is taken over what that authentication established.

14.2 Possession and relationship are different predicates

A possession predicate has the general form

$$\text{Possess}(x, k),$$

meaning that executor x controls artifact k .

A Proof-of-Possession establishes some proposition of that form, or a closely related holder-binding proposition:

$$\text{PoP}(x, k).$$

It does not by itself establish $q_J(x) = u$. Inferring profile membership requires a trusted binding from k or its credential to the executor and to the asserted class attributes. Conversely, a sound profile-evidence mechanism may identify and classify the executor by boundary-local observation without requiring successor-side PoP.

Nor does possession determine causal history. Two occurrences may have the same executor, key, credential, authenticated profile and possession evidence while belonging to different authorization-aware histories. Neither $\text{Lineage}(e) = \hat{\ell}$ nor $\text{PoR}(s_i, s_{i+1})$ follows from $\text{Possess}(x, k)$.

A Proof of Relationship establishes something categorically different:

$$\text{PoR}(s_i, s_{i+1}),$$

namely that one execution occurrence validly continues another.

The two propositions may be connected operationally, but neither is logically identical to the other.

14.3 Evidence is not semantics

A concrete implementation may use a signature, key, credential, attestation, trusted execution environment, authenticated channel, capability handoff, or another mechanism to provide evidence for one of the following facts:

$$q_J(x) = u,$$

$$\rho(\tau) = 1,$$

or

$$\text{PoR}(s_i, s_{i+1}).$$

PoP may be one such evidence mechanism. For example, possession of a predecessor-issued continuation artifact may witness a single causal handoff. But the authorization semantics consumes the proved relationship and the monotone state; it need not consume possession itself.

Definition 17 (Possession-independent relational authorization). *A relational authorization rule is possession-independent when its logical premises contain no predicate of the form*

$$\text{Possess}(x, k)$$

or

$$\text{PoP}(x, k).$$

Proposition 8 (Semantic possession-independence). *Once $\text{PoR}(s_i, s_{i+1})$, the executor profiles, the relational predicate verdict and the two monotonicity conditions are given, the relational authorization rule of Definition 9 is possession-independent in the sense of Definition 17.*

Proof. The rule’s premises are exactly $\text{PoR}(s_i, s_{i+1})$, $C_{i+1} \subseteq C_i$, $\rho_i(\tau_i)$, and $\rho_{i+1} \Rightarrow \rho_i$. None is a predicate of the form $\text{Possess}(x, k)$ or $\text{PoP}(x, k)$.

This proposition records an immediate semantic fact about the abstract rule; it does not show that its premises can be soundly realized without successor-side PoP. That separate, substantive question is addressed by the conditional construction below.

Proposition 6 places successor-side possession, where it is used, in the authentication layer. The construction below shows that even that layer can be discharged without it. This matters for the non-necessity claim: the rule would be empty if every sound construction of PoR required the successor to prove possession of something: a non-necessity result has content only if its premises are satisfiable without the excluded predicate. We therefore give a conditional construction in which the successor proves possession of nothing, and state rather than conceal the environmental assumptions on which it depends.

14.4 A construction without successor-side possession

Let h be a collision-resistant commitment to the state of an execution step, and H a collision-resistant hash. Let each step s_i be accepted at an enforcement boundary B_i : the verifier that evaluates Valid_R for the hop entering s_i , and that holds a signing key. Write $\sigma_B(\text{enc}(m))$ for a signature by B on the canonical encoding of a structured payload m , and let λ be a lineage identifier fixed at the origin. All signed and hashed tuples use the same injective canonical encoding enc .

A checkpoint is the tuple

$$\kappa_i = (B_i, m_i, \sigma_{B_i}(\text{enc}(m_i))).$$

It is identified by its signer and signed payload, not by the bytes of its signature. Its identifier is

$$\text{cid}(\kappa_i) := H(\text{enc}(B_i, m_i)).$$

Signature schemes may be malleable, admitting several valid signatures on one payload; all of them have the same identifier, so malleability affects neither predecessor binding nor the consumption register.

Definition 18 (Boundary-witnessed checkpoint). *The checkpoint of step s_{i+1} is*

$$\begin{aligned} \kappa_{i+1} &:= (B_{i+1}, m_{i+1}, \sigma_{B_{i+1}}(\text{enc}(m_{i+1}))), \\ m_{i+1} &:= \left(\begin{array}{c} \text{cid}(\kappa_i), h(s_{i+1}), \lambda, \\ C_{i+1}, \rho_{i+1}, q_J(x_{i+1}), \nu_{i+1} \end{array} \right), \end{aligned}$$

where ν_{i+1} is a nonce generated by B_{i+1} . The boundary B_{i+1} issues κ_{i+1} only after:

1. verifying the predecessor checkpoint κ_i and that it carries the same lineage identifier λ ;
2. checking that $\text{cid}(\kappa_i)$ has not already been consumed as the predecessor of an accepted step;
3. observing that s_{i+1} consumes the continuation emitted by the step committed in κ_i , written $\text{ObservedHandoff}(s_i, s_{i+1})$;
4. identifying the executor x_{i+1} and determining its profile $q_J(x_{i+1})$; and
5. checking $C_{i+1} \subseteq C_i$, obtaining $E_{i+1} \vdash \rho_i(\tau_i)$, and checking $\rho_{i+1} \Rightarrow \rho_i$, where E_{i+1} is the evidence available to the receiving boundary, and where C_i , ρ_i and $q_J(x_i)$ are read from κ_i and $\tau_i = (q_J(x_i), q_J(x_{i+1}))$. Refinement is checked by the syntactic criterion of Section 7, which is sound.

The origin checkpoint uses the same payload with a distinguished origin marker $\perp$ in place of $\text{cid}(\kappa_{-1})$:

$$\begin{aligned} m_0 &= (\perp, h(s_0), \lambda, C_0, \\ &\quad \rho_0, q_J(x_0), \nu_0), \\ \kappa_0 &= (B_0, m_0, \\ &\quad \sigma_{B_0}(\text{enc}(m_0))). \end{aligned}$$

Authority bootstrap and the decision to issue κ_0 are out of scope as in [1].

Write $\text{stepCommitment}(\kappa_i)$ for the second component of the checkpoint payload, namely $h(s_i)$.

The nonce plays no challenge-response role: no successor answers it. It distinguishes checkpoints issued by the same boundary for otherwise identical steps. Since the identifier also covers the issuing boundary B_i , entries in the consumption register of the single-consumption/anti-replay assumption below are unambiguous across boundaries as well.

The construction yields a predicate

$$\text{PoR}_\kappa(s_i, s_{i+1}),$$

which holds iff κ_{i+1} verifies, its payload's first component is $\text{cid}(\kappa_i)$ for a verified κ_i , and both carry λ , with

$$\begin{aligned} \text{stepCommitment}(\kappa_i) &= h(s_i), \\ \text{stepCommitment}(\kappa_{i+1}) &= h(s_{i+1}). \end{aligned}$$

This constructed predicate is distinct from the semantic $\text{PoR}(s_i, s_{i+1})$, which holds iff s_{i+1} is a valid causal continuation of s_i . What must be shown is that PoR_κ implies PoR .

Binding $\text{cid}(\kappa_i)$ rather than only the commitment $h(s_i)$ ties each checkpoint to the exact predecessor payload, and with it to the context C_i , the envelope ρ_i , the profile $q_J(x_i)$ and the lineage λ that the checks in item 5 read. Except with negligible probability, a checkpoint cannot be spliced onto a different predecessor that happens to share a step commitment.

We work in the computational model. Collision resistance does not make H injective: collisions exist, but no efficient adversary finds one except with probability negligible in the security parameter η . Statements about uniqueness below therefore hold except with negligible probability, not absolutely.

The construction rests on seven separate assumptions.

Assumption 4 (Witness assumptions).

1. Unforgeability and binding. *Except with probability negligible in η , no efficient adversary forges a boundary signature or finds a collision in h or H . Verifiers possess authentic verification keys for the boundaries they trust.*
2. Protocol compliance. *Boundaries follow the protocol: a boundary issues a checkpoint only after performing checks 1–5 correctly, and records each consumed predecessor.*
3. Handoff observation and adequacy. *A boundary observes, by its own means, which continuation a step consumes. Observed consumption is the operational criterion of causal continuation:*

$$\text{ObservedHandoff}(s_i, s_{i+1}) \implies \text{PoR}(s_i, s_{i+1}).$$

4. Non-PoP profile observation. *A boundary identifies the executor it admits, and soundly determines its profile q_J , from boundary-local observation or independently authenticated infrastructure evidence, without requiring the successor to demonstrate control of any artifact. For example, the boundary may hold a trusted mapping from the execution slot, container or virtual machine it controls to the executor, and from the executor to its profile:*

$$\text{slot} \mapsto x_{i+1} \mapsto q_J(x_{i+1}).$$

5. Relational evidence soundness. *For every atom occurring in an admitted envelope, the boundary either computes the atom from verified profiles or obtains sound, monotone evidence for its truth in the sense of Section 6.2.*
6. Single-consumption / anti-replay. *A checkpoint identifier is accepted as the predecessor of at most one step, and no boundary reuses a nonce. This requires state: a consumption register on which acceptance and marking are atomic, shared by the boundaries that may receive a given continuation, or a continuation that designates a single verifier with an atomic local register.*

7. Root validity. *The origin gate issues κ_0 only with*

$$\text{stepCommitment}(\kappa_0) = h(s_0),$$

a fresh lineage identifier and nonce, the origin contexts C_0, p_0 selected by the bootstrap policy, and a sound profile $q_J(x_0)$ for the executor of s_0 . The root tuple is canonically encoded and signed by an authenticated origin boundary. This assumes the truth of the root payload; a valid signature alone does not establish it.

“By its own means” means without relying on credentials presented by the executor; the paragraph after the proof explains why this matters.

A continuation may designate a receiving verifier without designating the future executor: the verifier is an enforcement component, whereas x_{i+1} may remain unknown until the continuation is presented for admission.

The single-consumption clause is stronger than the core PIC transition semantics. Definition 9 does not prescribe one successor per checkpoint. The present witness construction adds the property that a predecessor checkpoint has at most one accepted successor.

This is a transition-admission property, not a message-delivery semantic. Duplicate or at-least-once delivery is compatible with the same guarantee of at most one accepted successor when repeated presentations of the same predecessor are deduplicated by the atomic consumption decision. Conversely, a realization may deliberately admit several successors, as in fan-out; such a realization does

not provide the witness construction’s property that the checkpoint has at most one accepted successor.

In every case, each accepted continuation must independently satisfy the four PIC transition obligations.

Single consumption does not require the predecessor to know the concrete successor in advance. In the $N+1$ case, x_{i+1} may not yet exist, be selected, or be provisioned when s_i emits its continuation. The uniqueness decision is therefore made at admission: a valid candidate continuation is evaluated by the receiving boundary and, in a single-consumption realization, acceptance atomically consumes the predecessor. A realization that instead required s_i to pre-bind the continuation to the future successor’s identity, workload key, holder identity, or communication channel would not cover the unknown-successor case considered by the model.

Proposition 9 (Relationship without successor-side possession). *Under Assumption 4, and except with probability negligible in η , $\text{PoR}_\kappa(s_i, s_{i+1})$ implies $\text{PoR}(s_i, s_{i+1})$. Moreover, if κ_{i+1} verifies, then κ_i has at most one accepted successor, $q_J(x_{i+1})$ is the profile of the executor that realized s_{i+1} , and the conditions $C_{i+1} \subseteq C_i$, $\rho_i(\tau_i)$ and $\rho_{i+1} \Rightarrow \rho_i$ hold. Verifying κ_{i+1} involves no predicate $\text{Possess}(x_{i+1}, k)$ for any artifact k .*

Proof. Suppose $\text{PoR}_\kappa(s_i, s_{i+1})$. By unforgeability, except with negligible probability, B_{i+1} issued κ_{i+1} . Its payload’s first component is $\text{cid}(\kappa_i)$ for a verified κ_i . A verified checkpoint on the same payload but with different signature bytes has the same identifier, and is the same checkpoint for every purpose of the protocol. A verified checkpoint κ'_i on a different payload or from a different boundary with $\text{cid}(\kappa'_i) = \text{cid}(\kappa_i)$ would, since enc is injective, be a collision in H : any execution of the system that produced one would yield an efficient collision finder, so this happens with negligible probability. Except with negligible probability, the predecessor binding therefore cannot be redirected to a different valid checkpoint, and κ_{i+1} is bound to the values in κ_i . The two step-commitment equalities in the definition of PoR_κ , together with collision resistance of h , bind those checkpoints to the named arguments s_i, s_{i+1} except with negligible probability. For a chain rooted at κ_0 , root validity supplies the corresponding base bindings and sound origin payload. By induction on i , the predecessor values in κ_i are correct. By protocol compliance, B_{i+1} performed checks 1–5 before issuing. Check 3 together with handoff observation gives $\text{ObservedHandoff}(s_i, s_{i+1})$, and handoff adequacy gives $\text{PoR}(s_i, s_{i+1})$. Check 2 together with single-consumption/anti-replay gives that κ_i has at most one accepted successor. Check 4 together with non-PoP profile observation gives that $q_J(x_{i+1})$ is correct. Check 5 gives $C_{i+1} \subseteq C_i$ directly and $\rho_{i+1} \Rightarrow \rho_i$ by soundness of the syntactic criterion. For $\rho_i(\tau_i)$, relational evidence soundness and the lifting rules of Section 6.2 turn $E_{i+1} \vdash \rho_i(\tau_i)$ into $\rho_i(\tau_i)$. All three are checked against predecessor values bound in κ_i and successor values bound in κ_{i+1} . Verification reads κ_i, κ_{i+1} , the boundaries’ public keys, and

the boundary-local or infrastructure evidence permitted by Assumption 4; nothing in it is held or presented by x_{i+1} .

The substantive content of the proposition lies in handoff observation and adequacy. The construction converts an observation made by a compliant boundary into verifiable evidence; it does not create causal knowledge that the boundary did not have. Protocol compliance makes explicit what the previous paper leaves implicit when it places compromise of the validator out of scope [1].

Realizing the assumptions without successor-side possession. The proposition shows that verification reads nothing from the successor. That is not enough, because possession could re-enter through the assumptions themselves. In practice, handoff observation and executor identification are often realized with a key held by the workload, such as a SPIFFE SVID or an attestation key presented by the executor. That is a proof of possession by the successor. If the assumptions were realized that way, the claimed no-successor-PoP realization would be circular. This is why handoff observation is required to hold by the boundary’s own means, and why profile determination is required to be non-PoP profile observation. Two architectural patterns illustrate how this requirement may be met.

- *Launch and measure.* The boundary is the runtime that instantiates the executor. It launches x_{i+1} from an image it has measured, and assigns its profile at launch from that measurement and from policy. Identity and profile come from the launch record kept by the boundary, not from a key held by the workload.
- *Co-located observation.* A runtime co-located with the executor, such as a hypervisor, a sandbox supervisor, or a proxy through which all input reaches the executor, delivers the continuation into the executor and observes its consumption directly. It knows which executor it hosts because it hosts it.

In both, the executor presents nothing: the boundary knows what it observes because it controls the execution environment. Deployments that instead rely on workload-held keys use successor-side PoP. The model still applies to them, but the no-successor-PoP conclusion of Theorem 3 applies only when the Witness Assumptions are in fact discharged by the deployment.

Possession does not disappear from the system: the boundaries control signing keys. In this construction it moves from the successor to the witnesses, from the presenter of authority to the root of trust that attests the causal handoff. The successor is witnessed; it presents nothing. Verification of a witness signature establishes the source and integrity of the attestation under the trusted key; it does not by itself make the attested handoff or profile true. Those conclusions use handoff adequacy and

profile soundness. The authorization rule then evaluates PoR, C_i , ρ_i , and the established profile facts.

The same boundary also signs the profile facts $q_J(x_{i+1})$, in the same checkpoint and under the same key. Relational authorization therefore requires no additional checkpoint-signing authority, provided the boundary that PoR already trusts is also trusted to determine the profile facts. It does not follow that no further trust is needed. Signing a statement does not make it true, and the underlying profile evidence, such as the registry of measured images, the hardware root of the launching platform, or the organizational authority that assigns classes, may depend on additional trust roots. The non-PoP profile observation assumption absorbs them.

Theorem 3 (Conditional realization without successor-side possession). *Under Assumption 4, and except with probability negligible in η , the boundary-witnessed checkpoint construction realizes the premises of Proposition 8 for every transition it admits, without requiring any successor executor to prove possession of an artifact. Its admitted transitions therefore form a PIC-compatible realization in the sense of Definition 10. Boundary signing keys provide checkpoint evidence; external profile evidence may rely on additional trust roots. This conclusion is relative to the Witness Assumptions, not an unconditional proof that every required environmental observation can be realized in every deployment.*

Proof. Proposition 9 gives, under Assumption 4, a construction in which PoR_κ soundly witnesses PoR, the successor profile is sound, and $C_{i+1} \subseteq C_i$, $\rho_i(\tau_i)$, and $\rho_{i+1} \Rightarrow \rho_i$ hold. Relational evidence soundness lifts the verified atomic facts to the admitted positive predicate. These are exactly the requirements of Definition 10. Handoff observation by the boundary’s own means and non-PoP profile observation are hypotheses, so no successor proves possession of anything within the construction. The resulting admission rule is possession-independent by Proposition 8.

The launch-and-measure and co-located patterns above are architectural examples, not a formal witness model proving the joint satisfiability of Assumption 4. Accordingly, the checkpoint argument is a proof sketch in the computational model whose soundness is conditional on the seven listed assumptions.

Theorem 3 concerns admission of a transition by Valid_R . It yields a statement about an occurring resource effect only when complete mediation connects effects to accepted steps; claims about effects caused by a request additionally use causal attribution, exactly as in Theorem 2.

14.5 What the result does and does not claim

Proposition 8 records the semantic non-necessity of PoP in the abstract rule. Theorem 3 is the separate, conditional

realization result. Neither claims that PoP has no security role.

PoP may still be useful for:

- proving control of a private key;
- holder binding;
- authenticating an evidence producer;
- proving control of a continuation artifact;
- protecting transport or replay-sensitive mechanisms; and
- implementing a particular PoR construction.

The distinction is semantic:

PoP may provide holder-binding or authentication evidence;

whereas

PoR establishes causal continuation; C_i bounds authority and ρ_i bounds relational continuation.

Within this model, possession presented by the successor is therefore an optional evidence or authentication-layer mechanism rather than a necessary premise of the authorization rule. The conditional witness construction places cryptographic possession at the boundary rather than at the successor; other deployments may use PoP at either layer without making it a semantic premise.

Remark 2 (Mechanism independence). *Outside the witness construction of Section 14.4, which is one realization among many, no result in this paper refers to cryptography. PoR, profile membership and the monotone state enter only as predicates; signatures, hashes, attestation and proofs of possession are ways of producing evidence for them. Proposition 7 proves invariance under replacement of a profile-evidence mechanism only when the replacement is sound and supplies the same profile. It does not prove that arbitrary evidence mechanisms are interchangeable, nor does it remove the separate soundness obligation for evidence of PoR or relational atoms. PoR and PoC are modeled as properties of the execution; PoP is one possible property established by an evidence protocol.*

15 From Possession to Relation

A possession-centered authorization rule can be abstracted as

$$\text{Possess}(x, a) \wedge \text{Grants}(a, o, r) \implies \text{Authorized}(x, o, r).$$

The relational model instead evaluates an execution occurrence:

$$\text{PoR}(s_i, s_{i+1})$$

together with monotone state evolution

$$\Gamma_{i+1} \preceq \Gamma_i.$$

The authorization question therefore changes from:

What authority does this actor possess?

to:

Is this occurrence a valid continuation of an already authorized causal relation, under an authorization state that has not expanded?

This does not eliminate identity, credentials, or possession mechanisms. Rather, it relocates them. Identity and PoP may establish facts about the executor or the evidence; the authorization relation is evaluated over the causal and relational structure of the execution.

16 A Realization of the Premises: PIC-X

The results of this paper are stated over abstract predicates and do not depend on any implementation. This section records how one open-source implementation, PIC-X [6], part of the PIC protocol [2], discharges the restricted subset of premises identified below. Its purpose is an engineering illustration, not a proof or a complete witness of the model. The description was checked on 25 September 2026 against the PIC-X source prepared for release v0.2.3, which builds on the `pic-protocol` crate, version 0.2.3. The PIC-X release tag is an implementation snapshot, not an assumption of any theorem in this paper.

PIC-X takes the layered route of Proposition 6, not the witness route of Section 14.4. The successor authenticates with possession, and each checkpoint is signed by the realm that settles the transition. Possession-based signatures have two evidence-producing roles: they support workload/profile authentication through the attester binding, and they demonstrate control of the bytes used to advance a continuation artifact. The predicate Possess itself is not a primitive of the abstract relational authorization decision.

A difference in terminology. PIC-X uses the name “Proof of Relationship” for the evidence that authenticates the workload: an SD-JWT [7] issued by an attester that the realm trusts, which binds the workload’s public key through its confirmation claim. In the terms of this paper, that evidence belongs to the authentication layer. The causal relationship that this paper calls PoR is only partially evidenced in PIC-X by the transition, which references the exact predecessor checkpoint; this does not establish an observed handoff. PIC-X therefore describes this implementation level as *artifact-linked*: the label records the strength of the available evidence and is not a new semantic definition of PoR.

Settlement. The workload submits a candidate transition and signs it, together with the candidate continuity and the candidate token, with the key that the attester bound. The realm then verifies, in order: the attester’s presentation and the workload’s three signatures; that the predecessor checkpoint verifies under the realm’s own key; the hash of the exact predecessor bytes, the position progression, the challenge chain, the lineage identifier and the expiry; the conformance of the disclosed claims to the execution contract; non-expansion; the configured revocation hook, which is currently a no-op; and local policy. Only then does it issue the next checkpoint, signed by the realm.

The execution contract as a unary envelope. The execution contract is a restricted envelope. Its atoms depend only on the successor’s profile, $\rho_i(u, v) = \bigwedge_k R_k(v)$, and they are combined by conjunction only. Addition-only accumulation is exactly the syntactic refinement $\rho_{i+1} = \rho_i \wedge \sigma$ of Section 7. The contract is evaluated from the predecessor checkpoint, which matches the design choice that ρ_i governs hop i . Atoms that relate predecessor and successor, such as the separation-of-duty atom of Section 18.3, are beyond this version.

Why positive predicates suit selective disclosure. The evidence monotonicity of the positive language has a direct reading here. A holder that withholds a disclosure removes evidence, and removing evidence cannot produce an admission that full disclosure would deny. Selective disclosure therefore cannot be used to widen admission. PIC-X behaves this way: a contract entry that is not disclosed is a rejection. The same property explains why constraints must be positive attested claims: an SD-JWT cannot establish the absence of a claim by withholding it.

Partial profiles. Selective disclosure makes the verified profile partial: the boundary learns only the components that were disclosed, while $q_J(x)$ in the model is total. The gap is the one the evidence relation $\vdash$ addresses. The boundary admits on $E \vdash \rho_i(\tau_i)$, where E contains the verified disclosures. Soundness of E gives $\rho_i(\tau_i)$ for the true profile, and undisclosed components can only prevent admission.

Where the realization departs from the model. The following departures affect no theorem, but they bound what PIC-X v0.2.3 realizes.

- *Successor-side possession.* PIC-X authenticates the successor with a key it holds. This is the layered route of Proposition 6: possession enters two evidence layers, authentication and continuation evidence, but not the decision rule. The conditional no-successor-PoP realization of Theorem 3 is the construction of Section 14.4, not PIC-X.

- *Fan-out.* PIC-X admits sibling branches from one checkpoint, while this paper models linear chains. Every root-to-leaf path is a linear chain, and every result of this paper applies along it. No branch can import authority from another, because each successor is attenuated from its own predecessor. The single-consumption/anti-replay assumption of the witness construction does not hold in PIC-X, and is needed only for the claim that a checkpoint has at most one accepted successor, which PIC-X deliberately does not provide.
- *Relationship as possession of the checkpoint.* PIC-X does not observe a handoff. The relationship is evidenced by the workload’s signature over a transition that references the exact predecessor bytes and continues its challenge. The checkpoint travels inside the settled token, and its challenge is in clear. Any workload that is attested by a trusted attester, conforms to the execution contract and obtains the settled token can therefore advance the lineage, whether or not the token was handed to it. The damage is bounded by the checkpoint’s authority and by the contract, but the relationship reduces to possession of a continuation artifact. The present stateless construction does not close this gap. In the models considered here, closing it requires trusted handoff observation, additional state, or an equivalent mechanism; the construction of Section 14.4 leaves that observation to a separate component.
- *No request binding.* The protocol crate offers a request-binding hook, but PIC-X’s artifact-linked policy explicitly accepts this check because no trusted request or transport occurrence is available at that boundary. A checkpoint is therefore not tied to the request it continues.
- *Unconstrained settled token.* The settled token carries no audience and no key binding. With an unknown successor this is unavoidable, but it makes the token a bearer credential for every attested, contract-conforming workload.
- *No revocation.* No revocation check is configured; a checkpoint is bounded only by the lineage expiry and by the retention of the realm key.
- *Attribution.* Complete mediation and causal attribution, the premises of Theorem 2 and Corollary 2, are properties of the deployment around PIC-X, such as resource servers that accept only settled tokens, not of PIC-X alone. Given the first item, PIC-X does not by itself supply causal attribution, including for dependencies carried through persistent state, asynchronous work, covert channels or side channels.

Planned implementation boundary as of 25 September 2026. These omissions are separated deliberately from the guarantees of v0.2.3. The planned next

Model element	PIC-X v0.2.3
Profile $q_J(x)$	Claims disclosed in the attester-issued SD-JWT presentation, verified at settlement and recorded with the attester’s identity for attribution.
Relation $\sim_j$	Not a separate object. Equality of the disclosed value of claim j , from a trusted attester, is the induced equivalence: the kernel of the claim function.
Authentication of membership	Attester-issued SD-JWT binding the workload key in its confirmation claim; the workload signs the transition, the candidate continuity and the candidate token with that key. Key-binding JWTs are not used, and are rejected.
PoR(s_i, s_{i+1}) in this paper	Only partially realized. The transition carries the SHA-256 hash of the exact predecessor checkpoint bytes, position $n + 1$, challenge continuity and the checkpoint’s lineage identifier, and the predecessor checkpoint must verify under the realm’s own key. This evidences possession of the predecessor checkpoint, not an observed handoff: handoff observation and adequacy are not realized.
C_i and $C_{i+1} \subseteq C_i$	Context of authority: identity context and invariants, attenuated by removal bitmaps and checked by subset inclusion.
ρ_i and $\rho_{i+1} \Rightarrow \rho_i$	Execution contract: key–value and membership constraints on the successor’s disclosed claims. The predecessor’s contract governs admission; additions are conjunctive only, and no entry can be removed or weakened. Unary and conjunctive; binary atoms and disjunction are not implemented.
Checkpoint signing	Each checkpoint is a COSE object signed by the realm; accepting a predecessor is a signature check under the realm’s published keys.
Freshness	SD-JWT validity claims, with bounded clock skew. The absolute lineage expiry is fixed in the initial checkpoint and preserved across advancement. A checkpoint is advanceable only before that expiry and while the realm key that signed it is published. The protocol crate provides a revocation hook at settlement, but PIC-X v0.2.3 configures none. No credential status list in this version.
Linearity and replay	Stateless: a checkpoint is not retired when a successor is issued, so fan-out is supported. There is no single-consumption register.

Table 1: Correspondence between the model and PIC-X v0.2.3, with `pic-protocol` 0.2.3.

implementation phase introduces trusted Execution Contexts at the consumption boundary. Because a message publisher may not know whether zero, one or many executors will consume a message, and a successor key may not yet exist, the design does not pre-bind a checkpoint to a future successor identity or key. Once a consumer is selected, an Execution Context can observe the concrete handoff and issue a signed receipt, or equivalent evidence, committing to the predecessor or candidate digest, a fresh occurrence identifier, the consumed request or message, and the locally authenticated executor profile. Authenticated channel facts may be included where the deployment can soundly observe them.

Such evidence is intended to strengthen artifact linkage into observed-handoff evidence, provide request binding, and support causal attribution for explicit mediated handoffs. Complete mediation additionally requires the protected resource and execution boundaries to reject effects not admitted through that path; PIC-X alone cannot establish it for hidden state, covert channels, or side channels. Optional replay protection, if introduced, must be scoped to an observed occurrence or delivery rather than globally retiring the predecessor, so that intentional fan-out remains possible. Revocation remains a separate mechanism, with explicit state and availability semantics. Per-claim attester trust, rather than the current issuer-wide trust, is independent hardening. None of

these planned mechanisms is assumed to exist in v0.2.3, and none is used by the paper’s theorems.

What the realization does not change. No theorem of this paper depends on PIC-X. PIC-X realizes authority attenuation, a unary conjunctive envelope, and profile soundness resting on the configured attesters (issuer-wide in this version); the linear results apply per path. It does not realize handoff observation and adequacy, the witness’s single-consumption/anti-replay property, request binding, or revocation, binary relational atoms, disjunction, complete mediation, or side-channel attribution, so it does not discharge the Witness Assumptions of Section 14.4 nor, by itself, the premises of Theorem 2.

17 Related Work

Capability-, caveat-, and logic-based authorization systems can encode or consume contextual and history-sensitive facts, including expiry, lineage or predecessor identifiers, delegation ancestry, ambient facts, and externally attested continuation evidence. The distinction claimed here is therefore not raw policy-language expressiveness. This model makes causal history a primitive dimension of the authorization domain and proves authority and relational closure along authorization-aware histories.

A mechanism does not already have PIC semantics merely because it can encode the facts that PIC requires. It is a PIC-compatible realization only when those facts are soundly established and the corresponding causal-history, authority-attenuation, relational-admissibility, and refinement obligations are enforced at admission. Representability of PIC premises is therefore not equivalent to PIC semantics.

An artifact-centered system can reproduce the same decision when supplied with equivalent trusted causal facts. Doing so requires more than adding a lineage field: at each hop it must soundly establish an equivalent valid continuation and the profile facts used in τ_i , bind the state to the corresponding history, check $C_{i+1} \subseteq C_i$, evaluate $\rho_i(\tau_i)$, and check $\rho_{i+1} \Rightarrow \rho_i$. These are exactly the obligations of Definition 10. Bare artifact possession supplies none of them by itself. Both such realizations and the witness construction require trusted evidence for causal facts; the witness assumption $\text{ObservedHandoff} \Rightarrow \text{PoR}$ does not create causal truth. The single-consumption/anti-replay rule of Section 14.4 supports that construction’s additional claim that a checkpoint has at most one accepted successor; it is not an extra conjunct of the core transition semantics.

A logic-based artifact can therefore consume an externally established `valid_continuation` fact. The remaining questions are what semantics makes that fact a sound witness of PoR, and whether admission enforces the two monotone obligations and relational admissibility. When those conditions hold, the artifact remains a capability carrier or policy engine, while the resulting authorization system is a capability-based PIC-compatible realization. This is a claim about the semantic structure of admission, not about greater raw expressiveness of one policy language over another.

Capabilities. Capabilities fuse designation and authority at invocation and thereby prevent the classical confused deputy when authority is exercised through the supplied capability rather than through ambient authority [9, 10]. In the terminology used here, such an invocation may realize a strong single-hop relationship between designation and authority.

The present model is compatible with capability implementations. A capability may be one way to establish a relational predicate, and an attenuated, context-bound capability may carry both authority and relational information. The distinction is not where the evidence is encoded but whether the authorization decision is invariant under changes in causal occurrence. An extended capability system that verifies a lineage-sensitive continuation condition can therefore enforce the same in-lineage non-mixing property. Claims about request-caused effects outside the represented lineage still require an equivalent of Complete Mediation and Causal Attribution; encoding the check in a capability does not supply those premises by itself. Capability mechanisms therefore remain valid

as representations and carriers of authority inside the broader causal-history authorization model.

Macaroons and caveated tokens. Macaroons attenuate a bearer credential by appending caveats, and caveats can confine authority by context, including by which party may exercise it [12]. Caveat chains are monotone in the same sense as C_i and ρ_i : adding a caveat can only restrict. A macaroon whose caveats bind the lineage and the executor class would therefore carry both monotone dimensions inside one artifact. In the terms of this paper it is a possession-based encoding of the envelope. Its continuity guarantees hold exactly to the extent that its caveats are lineage-sensitive, which is the condition identified in [1]. The present model states that condition independently of the artifact that carries it.

Attenuated delegation chains. Monotone attenuation along a chain is not itself new. Biscuit tokens carry Datalog checks, and the holder may append blocks that only add checks, restricting the token’s rights [13]. UCAN delegation chains require each delegation to restate or attenuate the capabilities of its delegator [14]. Two recent individual Internet-Drafts apply the idea to AI agents. One profiles OAuth JWT access tokens so that each delegation attenuates scope and constraints, with parent and child linked by a hash commitment and verifiable offline [15]. The other binds attenuated capability grants to encrypted task capsules, attenuating decryption access together with authorization [16]. Hash-linked predecessor binding, authority attenuation along a chain, monotone accumulation of restrictions and lineage verification are therefore shared with this line of work; the witness construction of Section 14.4 uses them and does not claim them as contributions. These systems attenuate delegated authority encoded in an artifact. The contribution here is a second ordered dimension, $\rho_{i+1} \Rightarrow \rho_i$ over relational transition profiles of executors, separate from the carrier of authority and composed with authority attenuation and causal continuity, together with the semantic separation from successor-side PoP.

Attribute-based access control. ABAC authorizes an operation by evaluating attributes of the subject, the object, the operation and the environment against policy [17]. The profile map q_J is an ABAC-style classification of executors, and atoms over $\mathcal{T}_J$ are attribute conditions over pairs of executors. What the model adds is not the classification but its propagation. An ABAC decision point evaluates the policy attached to the resource it guards, with no guarantee that a downstream decision point applies a policy no weaker than an upstream one. Here the envelope ρ_i is carried by the execution, bound to its lineage by PoR, and can only be refined along it. Relational closure is a property of the lineage, not of any single decision point.

Separation of duty. The NIST RBAC model standardizes dynamic separation of duty as a constraint on the roles a user may activate within a session [18], and history-based formulations make separation of duty depend on which principals have already acted [19]. The adjacent case of Section 18.3 is a lineage-scoped variant: the constraint is carried by the execution rather than attached to a session or an object, and no descendant can drop it. The non-adjacent case needs memory of earlier executors, which is the territory of the history-based formulations.

History-based access control. History-based access control likewise makes execution history relevant to authorization [11]. The construction here differs in emphasis: it treats executor equivalence classes and relational predicates as a first-class authorization space, then propagates that space monotonically along a PoR-linked discrete execution.

Proof-carrying and distributed authorization. In proof-carrying authorization, a requester presents a proof, in an authorization logic over credentials, that it is entitled to access, and a reference monitor checks the proof [20, 21]. The object of such a proof is the requester’s authority, typically rooted in credentials it holds or can obtain. The object of PoR is different: a relationship between two execution occurrences. The two are complementary. A proof-carrying system could serve as the mechanism that verifies the premises of Valid_R , and the lineage state (C_i, ρ_i) could appear as a premise in such a logic.

Scope of the quotient construction. The quotient construction is also deliberately agnostic to the concrete meaning of each equivalence relation. One relation may encode trust-domain membership, another a hardware-attestation class, another a workload class, and another an organizational relationship. Properties that depend on occurrences rather than executors, such as an execution-specific continuation property, do not belong in an equivalence relation on X ; they live in PoR or in the carried state. The authorization semantics depends only on the resulting quotient classes and relational predicates.

18 Discussion

18.1 Why quotient spaces matter

The quotient construction is not merely notation. It separates concrete identity from authorization-relevant identity.

Suppose $x \neq y$ but

$$x \sim_j y.$$

Then any policy intentionally defined only over

$$\mathcal{R}_j = X / \sim_j$$

must treat x and y equally. If the policy must distinguish them, then $\sim_j$ was too coarse and a finer relational view must be introduced.

Multiple quotient spaces therefore provide a disciplined way to state which executor differences matter to authorization and which do not.

18.2 Refinement instead of replacement

A lineage need not carry one fixed relational predicate forever. It may refine the predicate:

$$R_1 \vee R_2 \succeq_R R_1 \succeq_R R_1 \wedge R_3.$$

This is the relational analogue of authority attenuation:

$$\{\text{read}, \text{write}\} \supseteq \{\text{read}\}.$$

The shared principle is non-expansion.

18.3 Dynamic separation of duty

The relational envelope constrains which executors may act along a lineage, not only which authority they carry. It therefore expresses constraints between executors that authority attenuation alone cannot state.

Let $a, j \in J$. Let $\sim_a$ classify executors by role, and let $A \subseteq \mathcal{R}_a$ be the approver classes. For the relation $\sim_j$ and profiles $(u, v) \in \mathcal{T}_J$, with u_j the j -th component of u , define the atom

$$\text{SoD}_j(u, v) : \Longleftrightarrow v_a \notin A \vee u_j \neq v_j.$$

The atom is a fixed subset of $\mathcal{T}_J$ and is attested like any other. It holds trivially when the successor is not an approver; when it is, the approver must lie in a different $\sim_j$ -class from its predecessor.

A lineage whose proposal step s_i is followed by an approval step can refine its envelope at the proposal step to

$$\rho_i := \rho_{i-1} \wedge \text{SoD}_j.$$

This is a monotone refinement, checked by the receiving boundary at acceptance. Because every later envelope implies ρ_i , no descendant can drop the constraint. The result is dynamic separation of duty, the four-eyes principle, expressed as a refinement of the relational envelope.

Authority attenuation cannot state this constraint: proposer and approver may carry the same authority context, and the constraint concerns who exercises it. A pure bearer capability, without holder or context caveats, is monotone in authority and silent on the subject that exercises it, so the constraint is not a property of such a capability. Caveated tokens can express it, as discussed in Section 17. Here it is one refinement of ρ .

The atom relates adjacent executors, and the adjacent case lies within the formalism. The non-adjacent case does not. A predicate sees only $\tau_i = (q_J(x_i), q_J(x_{i+1}))$, and a proposer class recorded in a checkpoint is a state

variable outside that domain. A formal treatment extends the state to

$$\Gamma_i = (C_i, \rho_i, m_i),$$

where m_i is a write-once memory attested by the boundary, such as the proposer’s class, and lets predicates range over $M \times \mathcal{T}_J$. The closure arguments use only extension inclusion and transitivity, so they carry over to the extended domain unchanged. We leave that development to future work.

18.4 The role of causality

Neither authority attenuation nor relational refinement alone establishes continuity. The two sequences could be monotone but belong to unrelated executions.

PoR is what binds the monotone states to one causal history:

$$s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n.$$

Thus the model has three inseparable components:

1. causal linkage by PoR;
2. monotone privilege authority; and
3. monotone relational admissibility.

18.5 The lineage axis

Previous work represented an authority occurrence as

$$(o, r, \ell) \in O \times R \times \mathcal{L}_{\text{prev}},$$

where ℓ identifies the causal lineage. The present model does not replace that axis.

Instead, it refines the structure carried *along* a lineage. The lineage prefix

$$\widehat{\ell}_i$$

grows, while the attached state

$$\Gamma(\widehat{\ell}_i) = (C_i, [\rho_i]_{\text{sem}})$$

shrinks in the product attenuation order.

At the modeling level, this may be read as an ontological shift in what bears authorization state: the relevant object is not merely an executor or artifact, but an execution occurrence embedded in causal history. This is an interpretation, not an additional theorem; its mathematical content is the history-indexed state $\Gamma(\widehat{\ell}_i)$ and the transition rule of Definition 9.

It is therefore mathematically more precise to say:

continuity is monotone in the authorization state carried along the lineage, not that the lineage itself is decreasing.

19 Limitations and Future Work

The model is intentionally minimal and several questions remain open.

First, the equivalence relations $\sim_j$ are treated as given. A practical system must define who is authorized to assert them, how evidence is verified, how classes change over time, and how revocation affects class membership.

Second, the paper assumes a common relational vocabulary J . Heterogeneous systems may use different executor classifications at different boundaries. A general treatment would require sound translations between quotient spaces, analogous to authority translation between heterogeneous operation/resource spaces.

Third, the formal histories and proofs are linear. The core transition rule, however, does not prescribe single consumption: its obligations can be checked independently on each root-to-leaf path of a fan-out. The present witness adds at most one accepted successor per checkpoint and therefore excludes fan-out. Repeated delivery or presentation is compatible with this property when the atomic consumption decision deduplicates attempts. A fan-out realization instead deliberately admits several successors; each branch must independently satisfy the valid-transition obligations, but the realization does not provide the witness construction’s property that the checkpoint has at most one accepted successor. A unified treatment of forks, joins, fan-in and DAG-shaped executions still requires a partial-order generalization of both authority attenuation and relational refinement.

The same linearity excludes multiple-cause effects across distinct histories. Because Lineage is functional, Causal Attribution cannot hold simultaneously for $\text{CausedBy}(e, req_1)$ and $\text{CausedBy}(e, req_2)$ when $\widehat{\ell}_{req_1} \neq \widehat{\ell}_{req_2}$. Jointly caused effects require both a DAG or fan-in causal semantics and an explicit composition rule for the participating origin contexts and relational envelopes. Intersection of origin contexts is one possible conservative design, not a result of the present model; no composition rule is chosen here.

Fourth, PoR remains largely abstract. The paper proves properties conditional on sound verification of the causal relationship. The boundary-witnessed construction of Section 14.4 gives a no-successor-PoP realization relative to its Witness Assumptions, but does not independently prove their joint satisfiability. It is only a computational proof sketch and is not prescribed as the only construction.

Fifth, the exclusion of the confused deputy rests on complete mediation and causal attribution (Assumptions 1 and 2). The model does not decide whether an effect was caused by a request. The boundary-witnessed construction realizes attribution for explicit mediated hand-offs only; causality that flows through mutable memory, queues, databases, storage, caches, shared files, timers, asynchronous callbacks or background workers requires a stronger execution model, information-flow tracking for the modeled channels, or a further assumption. Covert

channels and side channels are not resolved by the witness construction: they may influence control or data flow without appearing as a mediated handoff, and ordinary information-flow tracking need not capture all of them. The confused-deputy result remains valid conditional on Causal Attribution, but the construction does not establish that assumption for arbitrary hidden channels. An error in attribution is a failure of the trusted computing base, not a violation of the continuity invariant.

The witness construction also separates assumptions that deployments must realize independently: hand-off observation, non-PoP profile observation, whose evidence may rest on further trust roots, and single-consumption/anti-replay.

A deployment that requires at most one accepted successor for a checkpoint needs an atomic consumption decision or equivalent coordination. When the consumption register is shared across boundaries, a globally unique acceptance decision requires consistent coordination; under a network partition, preserving that safety property may require delaying or rejecting admission rather than allowing two boundaries to accept the same predecessor independently. This is a realization-level distributed consistency/availability trade-off, not a theorem of the abstract PIC semantics.

Delivery semantics are separate from transition-admission semantics. An at-least-once transport may present the same continuation multiple times while an atomic consumption register deduplicates those attempts and still yields at most one accepted successor. Conversely, a realization may deliberately support fan-out and admit multiple valid successors. Such a realization retains the PIC obligations on each admitted branch but does not provide the witness construction’s property that the checkpoint has at most one accepted successor.

The N+1 unknown-executor property is orthogonal to this choice. The concrete successor need not be known when the predecessor acts; a single-consumption realization is compatible with that model provided uniqueness is enforced at admission without requiring pre-identification or pre-binding of the future executor. A continuation may designate a verifier without designating that future executor.

Dynamic separation of duty between non-adjacent steps requires the state extension sketched in Section 18.3.

Sixth, Proposition 8 is the immediate semantic result: PoP is not a premise of the abstract authorization rule once the required relationship facts are given. Theorem 3 is instead a conditional construction showing how those premises can be realized without successor-side PoP under the Witness Assumptions. A deployment may still choose PoP for engineering reasons, including authentication, key confirmation, replay resistance, or evidence transport.

The present model orders authorization state along an already materializing causal execution. A fuller temporal treatment must also account for executors, facts, and outcomes that do not yet exist when an earlier authorization

decision is made. The N+1 unknown-executor problem captures the case in which the concrete successor materializes only after the predecessor has acted, while the N+1 unknown-outcome problem captures the stronger case in which an authorization-relevant fact or concrete outcome itself becomes available only during execution [3, 4]. A future temporal authorization space may therefore need to represent admissible future occurrences or outcomes before their concrete witnesses materialize. We leave that extension to future work.

Finally, the model does not claim that every authorization policy should be relational. It identifies a relational authorization structure for discrete causal executions where authority continuity matters.

20 Conclusion

Proof-of-Continuity established that authority can be propagated through a causal execution by combining Proof of Relationship with monotone authority attenuation. This paper develops the relational structure implicit in that model.

Executors are classified through authorization-relevant equivalence relations:

$$\sim_j \subseteq X \times X.$$

Each relation induces a quotient:

$$\mathcal{R}_j = X / \sim_j,$$

which identifies physically different executors whenever their differences are irrelevant to the selected authorization characteristic.

Multiple relational quotients coexist, and their combined profile

$$q_J(x)$$

provides the relational coordinates of an executor. Predicates over transitions between such profiles compose with conjunction and disjunction, and implication defines an attenuation order:

$$\rho_{i+1} \Rightarrow \rho_i.$$

A continuous execution therefore carries two monotone authorization dimensions:

$$C_{i+1} \subseteq C_i$$

and

$$\rho_{i+1} \Rightarrow \rho_i.$$

At the same time, causal history accumulates:

$$\widehat{\ell}_i \preceq_L \widehat{\ell}_{i+1}.$$

The resulting structure is:

history grows while authority and relational freedom shrink.

Dual monotonicity gives combined closure. No valid descendant may introduce a privilege absent from the origin authority context, and no valid hop may introduce a relational transition excluded by the origin relational envelope. Under two explicit assumptions, complete mediation and causal attribution, the request-bounded-effect theorem excludes any accepted request-caused effect outside the origin authority context, and the confused-deputy result follows as a corollary. This does not preclude a buggy deputy from attempting such an operation; it precludes acceptance of the resulting effect under the stated premises. The enforcement boundary construction covers attribution only for explicit handoffs under its Witness Assumptions, and not for arbitrary state, covert or side channels. The authorization model is at the same time strengthened with relational closure.

PIC therefore separates authority possession from authority admissibility. A deputy may hold authority from unrelated sources, but that authority cannot become a valid exercise of the current lineage merely because the deputy executes one of its steps. Confused-deputy authority mixing is excluded by construction within a valid lineage; global exclusion of request-caused effects additionally depends on complete mediation and causal attribution. This is a semantic claim, not an exclusivity claim: other authorization mechanisms can obtain the same no-mixing result by adopting an admission rule that preserves the same lineage-local authority and relational constraints. Definition 10 makes this criterion explicit.

Most importantly, Proposition 8 records that the logical authorization rule contains no necessary successor-side possession premise. If PoR and relational-profile facts are soundly verifiable, PoP may supply holder-binding evidence or be used by one particular witness mechanism, but it is not required as the semantic basis of transition admission. Theorem 3 separately shows that the checkpoint construction supports a no-successor-PoP realization only relative to its explicit Witness Assumptions. Claims about resource effects additionally require complete mediation and causal attribution.

The distinction can be stated compactly:

possession can evidence control;
PoR establishes causal continuity.

Equivalently, in terms of the two layers:

sound evidence establishes
executor profiles;
PoR establishes causal continuation;
authorization combines PoR,
the profiles, C_i , and ρ_i to decide
what the occurrence may exercise
in that history.

The relevant authorization subject is therefore an execution occurrence in causal history, not a holder or artifact

alone. PIC does not replace capability mechanisms as representations or carriers of authority; it changes the semantic space in which carried authority may be exercised. Capabilities carry authority; PIC additionally governs whether this execution occurrence, in this causal history, may exercise that authority. Evidence used to establish PoR is not the semantic relationship itself, just as evidence used to establish a profile is not the authorization decision.

Under Definition 10, systems built from capabilities, Macaroons, Biscuit, UCAN or similar artifacts are PIC-compatible when they soundly establish equivalent causal-continuation and profile facts and enforce the same monotone lineage obligations. The artifact remains their authority carrier or policy engine; the semantics governing its exercise is PIC-compatible. This is why such a construction is a realization of the broader causal-history authorization semantics rather than a counterexample to it.

PIC therefore accommodates capabilities, credentials, caveats and PoP as evidence mechanisms without reducing authorization to their possession. It changes the domain in which their evidence is consumed: authorization applies to execution occurrences situated in causal histories. Consequently,

possessing authority
 $\neq$
being the occurrence permitted to exercise it
in this causal history

Acknowledgment of Prior Model

This paper extends the Proof-of-Continuity and Provenance Identity Continuity model developed in earlier work [1]. The new contribution is the executor quotient construction, the relational predicate order, dual monotonicity, relational closure, the reading of the earlier projection result as a quotient instance, and the possession-independence result together with a conditional construction that satisfies its premises without successor-side possession when the Witness Assumptions hold.

A Worked Example

Let $J = \{\text{org}, \text{role}, \text{prin}\}$, where $\sim_{\text{prin}}$ classifies executors by the principal on whose behalf they act. Several executor instances may share a principal, so its classes are not singletons. Profiles are triples $u = (u_{\text{org}}, u_{\text{role}}, u_{\text{prin}})$, and the approver classes are $A = \{\text{approver}\} \subseteq \mathcal{R}_{\text{role}}$. Define the atom

$$\text{SameOrg}(u, v) :\iff u_{\text{org}} = v_{\text{org}},$$

and take SoD_{prin} from Section 18.3. Both atoms are decidable from the profiles.

The lineage. A request enters at s_0 , realized by x_0 with profile (acme, requester, alice), with

$$\begin{aligned} C_0 &= \{(\text{read}, r), (\text{write}, r), \\ &\quad (\text{approve}, p)\}, \\ \rho_0 &= \text{SameOrg}. \end{aligned}$$

Step s_1 is realized by x_1 with profile (acme, approver, bob). The hop from s_0 is admitted, since ρ_0 holds on $(q_J(x_0), q_J(x_1))$. At s_1 , bob proposes a change and refines the envelope:

$$C_1 = C_0, \quad \rho_1 = \text{SameOrg} \wedge \text{SoD}_{\text{prin}}.$$

The refinement $\rho_1 \Rightarrow \rho_0$ holds syntactically, by conjunction. Step s_2 is an approval by executor x_2 . It presents evidence, for instance an SD-JWT from an issuer trusted for organization, role and principal, disclosing $\text{org} = \text{acme}$, $\text{role} = \text{approver}$ and $\text{prin} = \text{carol}$. The boundary verifies it and fixes $q_J(x_2) = (\text{acme}, \text{approver}, \text{carol})$.

Acceptance. With $\tau_1 = (q_J(x_1), q_J(x_2))$, the boundary checks $\rho_1(\tau_1)$. The organizations agree. The successor is an approver, and its principal differs from the predecessor's, since $\text{bob} \neq \text{carol}$. Both conjuncts hold. The boundary also checks $C_2 = \{(\text{approve}, p)\} \subseteq C_1$ and $\rho_2 = \rho_1$, a trivial refinement. The hop is accepted. By Proposition 7, any executor with the profile of x_2 , for instance another process acting for carol, would receive the same verdict, however it had authenticated.

Rejections. Four variants fail, each for a different reason.

1. Another executor instance acting for bob approves: $\tau = ((\text{acme}, \text{approver}, \text{bob}), (\text{acme}, \text{approver}, \text{bob}))$. The successor is an approver and the principals coincide, so SoD_{prin} fails. Separation is between principals, not instances: a second process of the same principal is not a second pair of eyes.
2. An approver from another organization: SameOrg fails.
3. An approver that withholds the organization disclosure: the evidence does not support SameOrg , so $E \not\models \rho_1(\tau_1)$ and the hop is rejected. By evidence monotonicity, no withholding can turn a rejection into an admission.
4. An approver whose transition carries $(\text{delete}, r) \notin C_1$: non-expansion fails.

In none of these cases does holding a valid credential change the outcome: the decision is taken over the profile, the authority context and the lineage.

In PIC-X v0.2.3, the fixed-value part of this policy can be written as an execution-contract entry such as $\text{org} = \text{acme}$. The relational atoms SameOrg and SoD_{prin} , which relate predecessor and successor, need binary atoms, which that version does not implement.

References

- [1] N. Gallo. Proof-of-Continuity: A Temporal Model for Authority Propagation in Distributed Systems and AI Agents. arXiv:2607.08906, 2026. <https://arxiv.org/abs/2607.08906>.
- [2] PIC Protocol. <https://pic-protocol.org/>. Accessed 25 September 2026.
- [3] PIC Protocol project. The N+1 Unknown Executor Problem. <https://github.com/pic-protocol/pic-protocol.github.io/blob/main/docs/why-pic/n-plus-1-unknown-executor-problem.md>.
- [4] PIC Protocol project. The N+1 Outcome on Unknown Facts. <https://github.com/pic-protocol/pic-protocol.github.io/blob/main/docs/why-pic/n-plus-1-outcome-unknown-facts.md>.
- [5] PIC Protocol project. The N+1 Invalid State Problem. <https://github.com/pic-protocol/pic-protocol.github.io/blob/main/docs/why-pic/authority-mixing.md>.
- [6] PIC Protocol project. PIC-X. Source code, version v0.2.3, built on the pic-protocol crate, version 0.2.3. <https://github.com/pic-protocol/pic-x>. Accessed 25 September 2026.
- [7] D. Fett, K. Yasuda, and B. Campbell. Selective Disclosure for JSON Web Tokens. RFC 9901, IETF, November 2025. <https://www.rfc-editor.org/info/rfc9901>.
- [8] N. Hardy. The Confused Deputy: Or Why Capabilities Might Have Been Invented. *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [9] J. B. Dennis and E. C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [10] M. S. Miller, K.-P. Yee, and J. S. Shapiro. Capability Myths Demolished. Technical Report SRL2003-02, Systems Research Laboratory, Johns Hopkins University, 2003.
- [11] M. Abadi and C. Fournet. Access Control Based on Execution History. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2003.
- [12] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrabie, and M. Lentzner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.

- [13] Eclipse Biscuit Project. Biscuit Authorization Token Specification. <https://github.com/eclipse-biscuit/biscuit/blob/main/SPECIFICATIONS.md>.
- [14] UCAN Working Group. User Controlled Authorization Network (UCAN) Specification. <https://github.com/ucan-wg/spec>.
- [15] R. Asor. Verifiable Attenuated Delegation for AI Agent Chains. Internet-Draft draft-asor-wimse-agent-delegation-chain-01, IETF, 3 September 2026. Work in progress. <https://datatracker.ietf.org/doc/html/draft-asor-wimse-agent-delegation-chain-01>.
- [16] H. Atakora. Capability-Bound Delegation Chains with Scoped Context Disclosure for AI Agents. Internet-Draft draft-atakora-wimse-sadp-delegation-01, IETF, 20 September 2026. Work in progress. <https://datatracker.ietf.org/doc/html/draft-atakora-wimse-sadp-delegation-01>.
- [17] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, 2014.
- [18] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli. Proposed NIST Standard for Role-Based Access Control. *ACM Transactions on Information and System Security*, 4(3):224–274, 2001.
- [19] R. T. Simon and M. E. Zurko. Separation of Duty in Role-Based Environments. In *Proceedings of the 10th IEEE Computer Security Foundations Workshop (CSFW)*, 1997.
- [20] A. W. Appel and E. W. Felten. Proof-Carrying Authentication. In *Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS)*, 1999.
- [21] L. Bauer, S. Garriss, and M. K. Reiter. Distributed Proving in Access-Control Systems. In *Proceedings of the IEEE Symposium on Security and Privacy*, 2005.
- [22] A. C. Myers and B. Liskov. A Decentralized Model for Information Flow Control. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 1997.